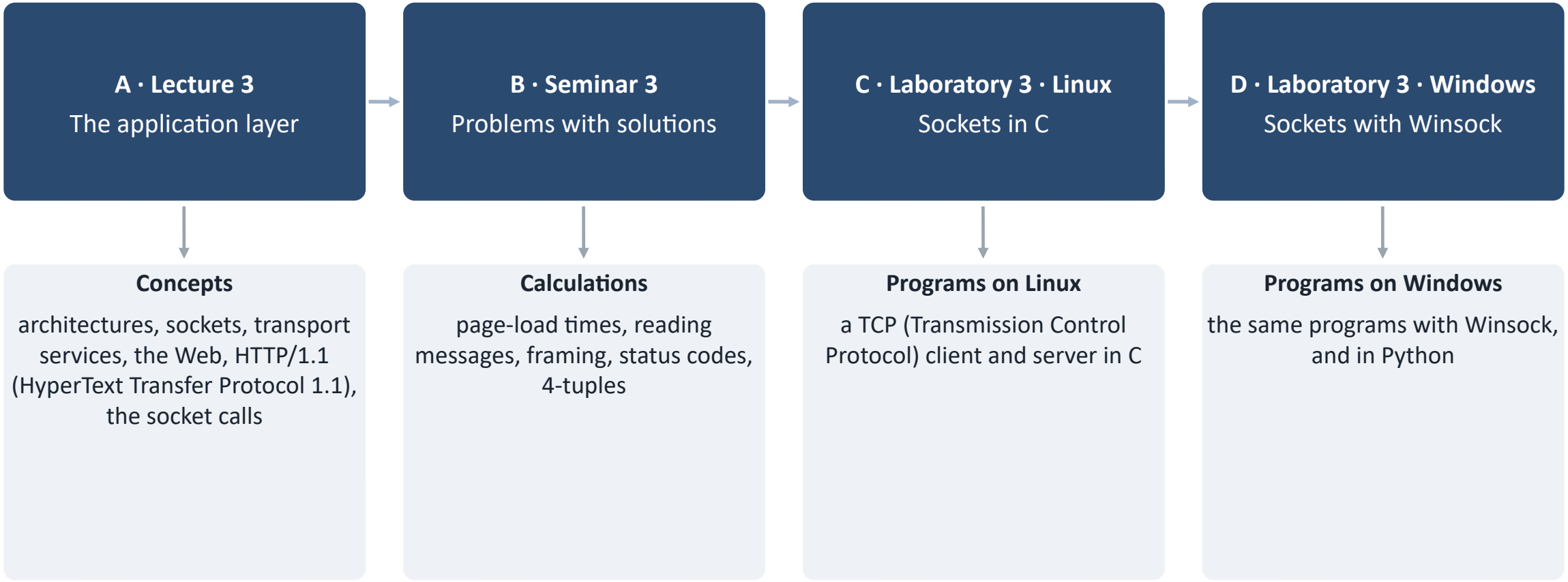
COMPUTER NETWORKS · SEMESTER 1 · WEEK 3

The application layer in schemes

Sockets, the Web and HTTP/1.1 — lecture, seminar and laboratories

Assoc. Prof. Nicolaie Popescu-Bodorin, PhD · 2026
+40722882471 · bodorin(on)live.com · https://lmrec.org/bodorin/

How this week is organised



Key idea Read the deck left to right: each part uses what the previous one built.

Part A · Lecture 3: the application layer

A · Lecture 3
The application layer

→

B · Seminar 3
Problems with solutions

→

C · Laboratory 3 · Linux
Sockets in C

→

D · Laboratory 3 · Windows
Sockets with Winsock

In this part

1 · Where the application layer sits

2 · Application architectures

3 · Processes, addresses, sockets

4 · What applications need from transport

5 · What a protocol defines

6 · The Web: objects and URLs

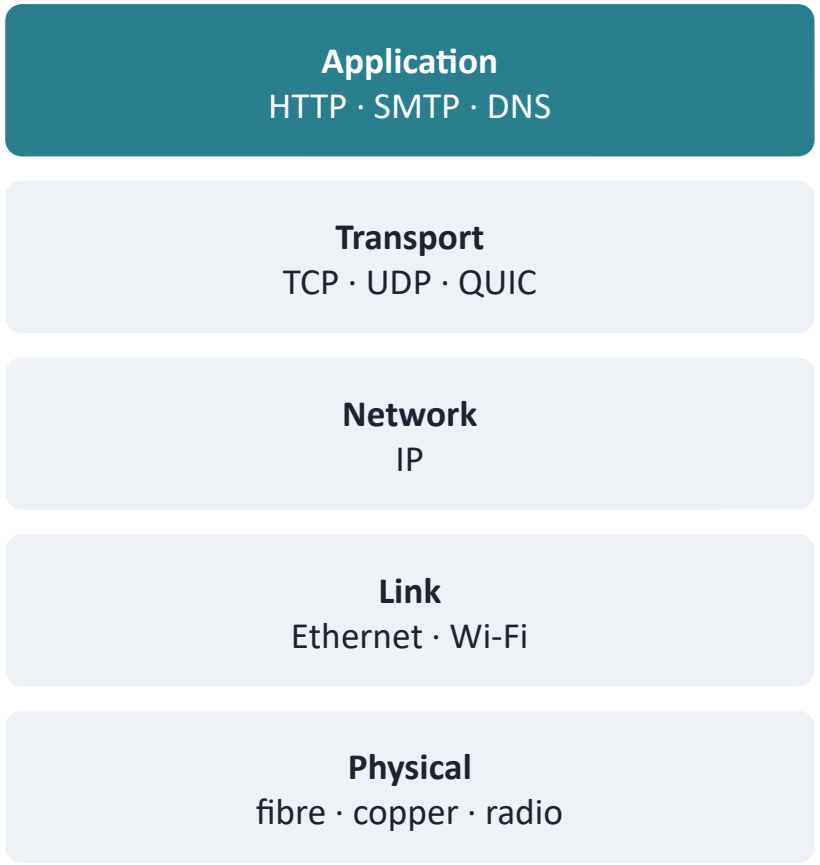
7 · HTTP connections and round trips

8 · HTTP/1.1 messages

9 · The socket calls

HTTP (HyperText Transfer Protocol)

1 · From the bottom of the stack to the top



1. **Application layer:** the protocols programs use directly — HTTP (HyperText Transfer Protocol), SMTP (Simple Mail Transfer Protocol), DNS (Domain Name System).
2. Below it: TCP (Transmission Control Protocol), UDP (User Datagram Protocol) and QUIC (a transport protocol over UDP; the name first stood for Quick UDP Internet Connections) carry its messages.

weeks 3 → 14: top-down

3. Network: IP (Internet Protocol). Link and physical: frames and signals.
4. This week: **the lecture** explains, **the laboratory** programs.

Key idea We study the layer you will program for the rest of your career first.

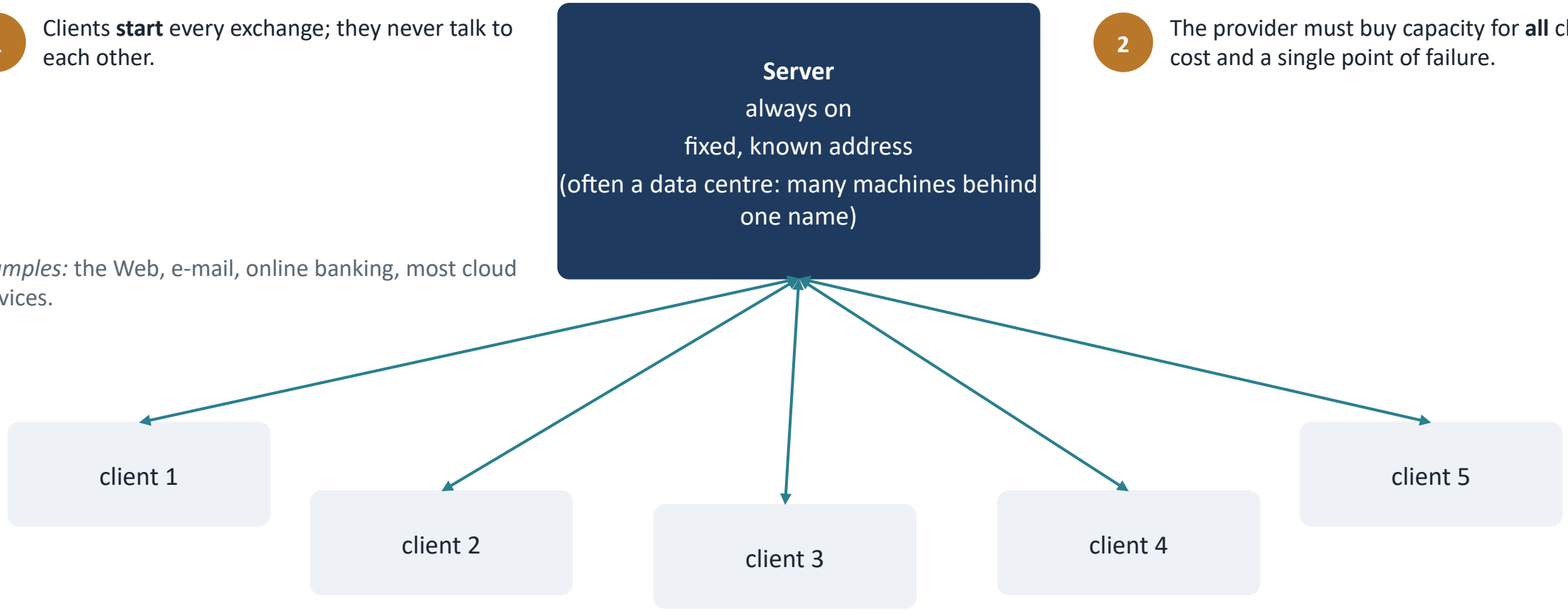
QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections)

2 · Client–server architecture

1 Clients **start** every exchange; they never talk to each other.

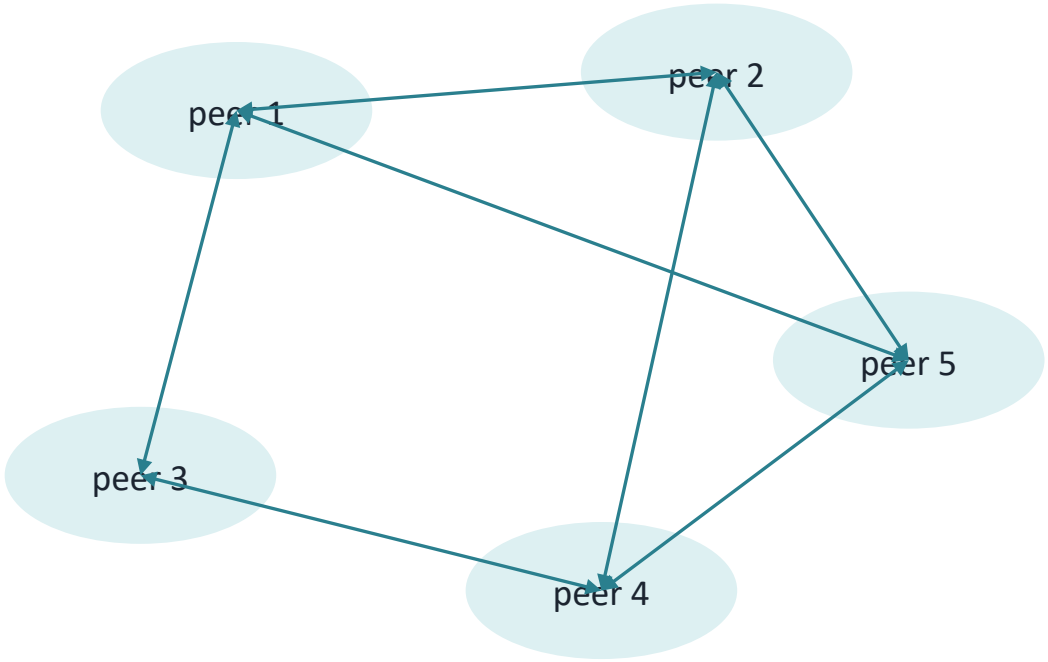
2 The provider must buy capacity for **all** clients: cost and a single point of failure.

Examples: the Web, e-mail, online banking, most cloud services.



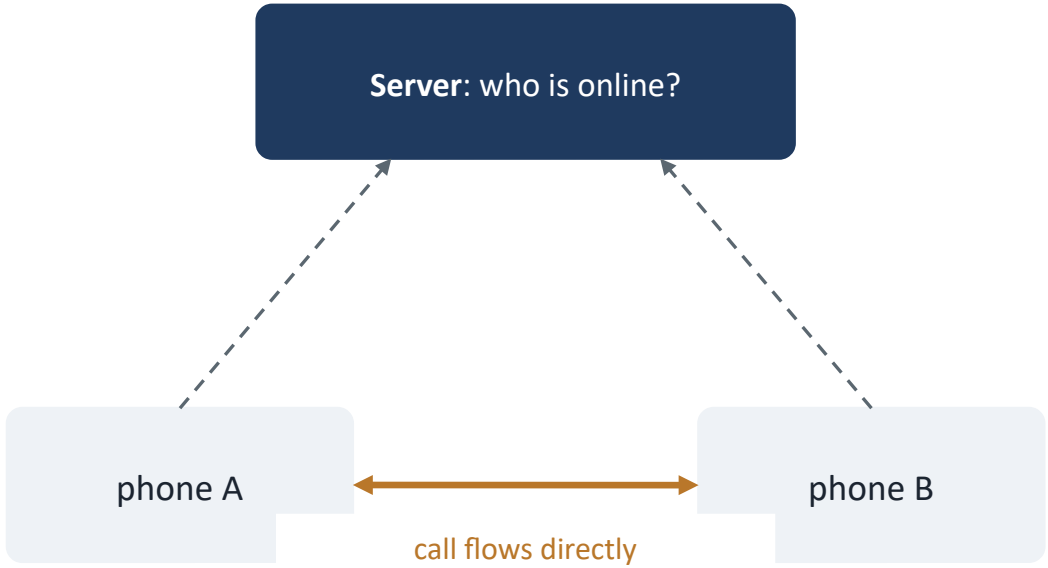
Key idea Client–server: one always-on side serves many clients — simple to control, expensive to scale.

2 · Peer-to-peer architecture and hybrids



P2P (peer-to-peer): every peer is client *and* server.

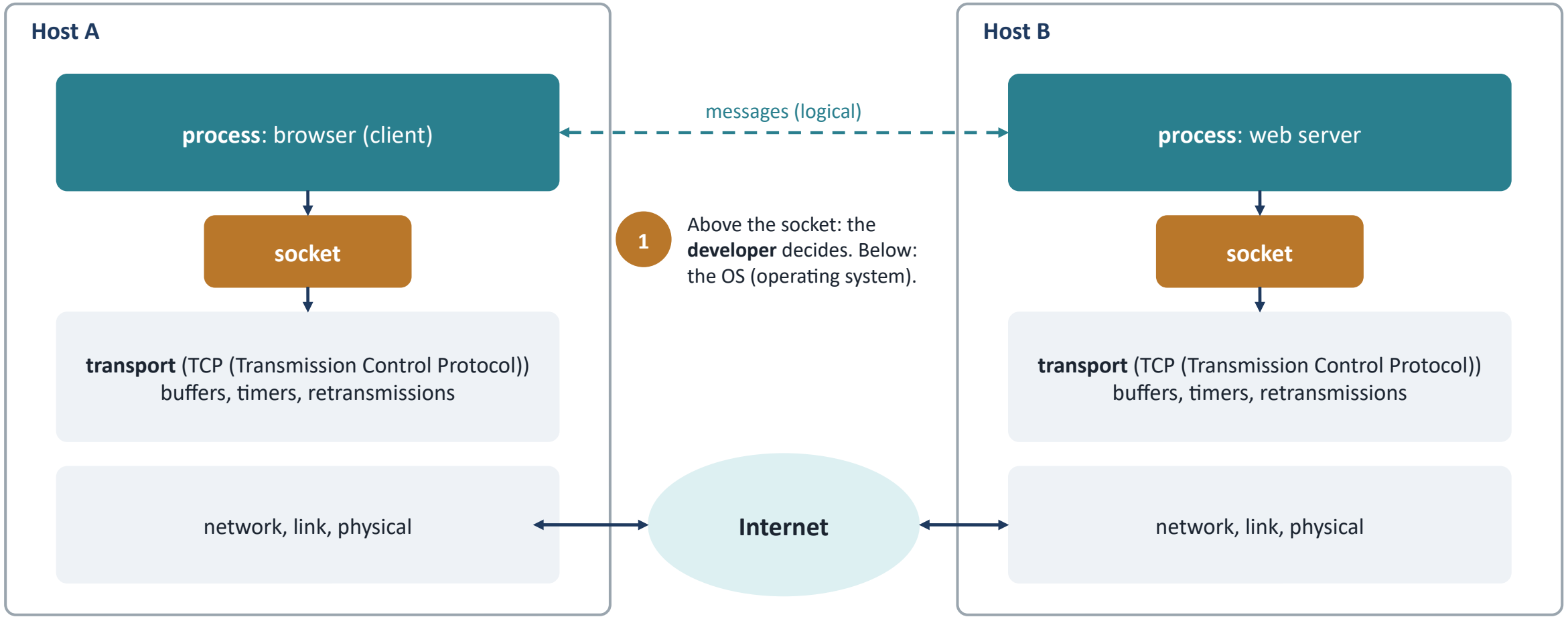
Self-scaling: each new peer brings demand *and* capacity.



1 **Hybrid:** a server finds the other user (dashed); the voice or video then flows peer to peer.

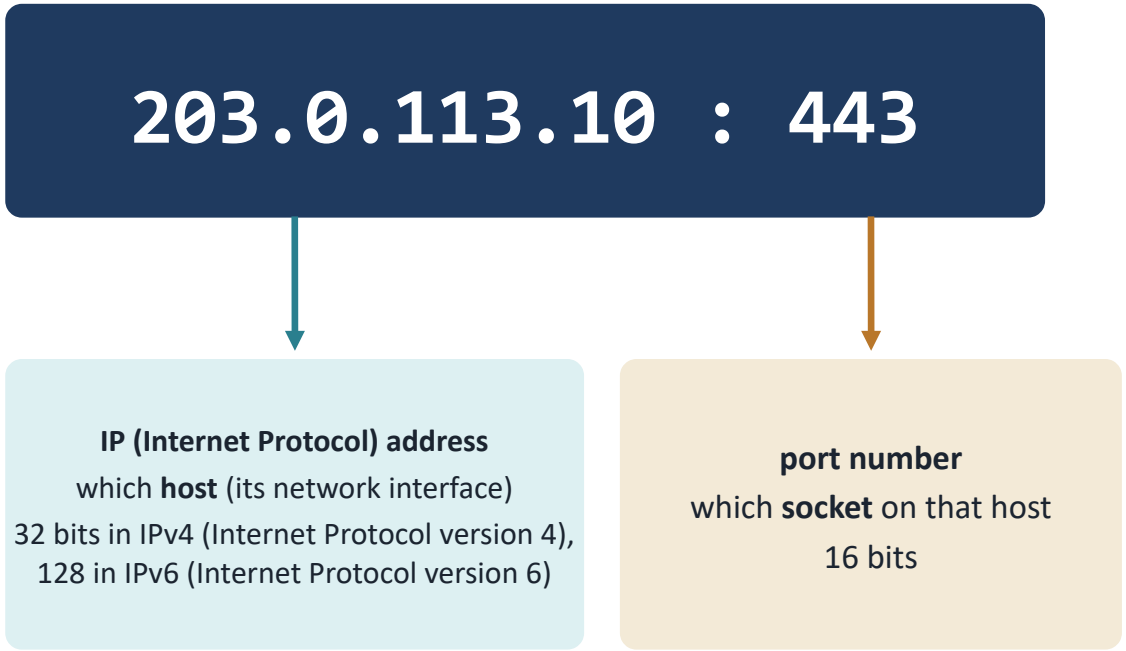
Key idea Peers scale themselves but come and go, sit behind routers and are hard to secure — so real systems mix both.

3 · Processes communicate through sockets



Key idea A socket is the door: the process pushes a message out and trusts transport to deliver it to the other door.

3 · Addressing a process: IP address + port



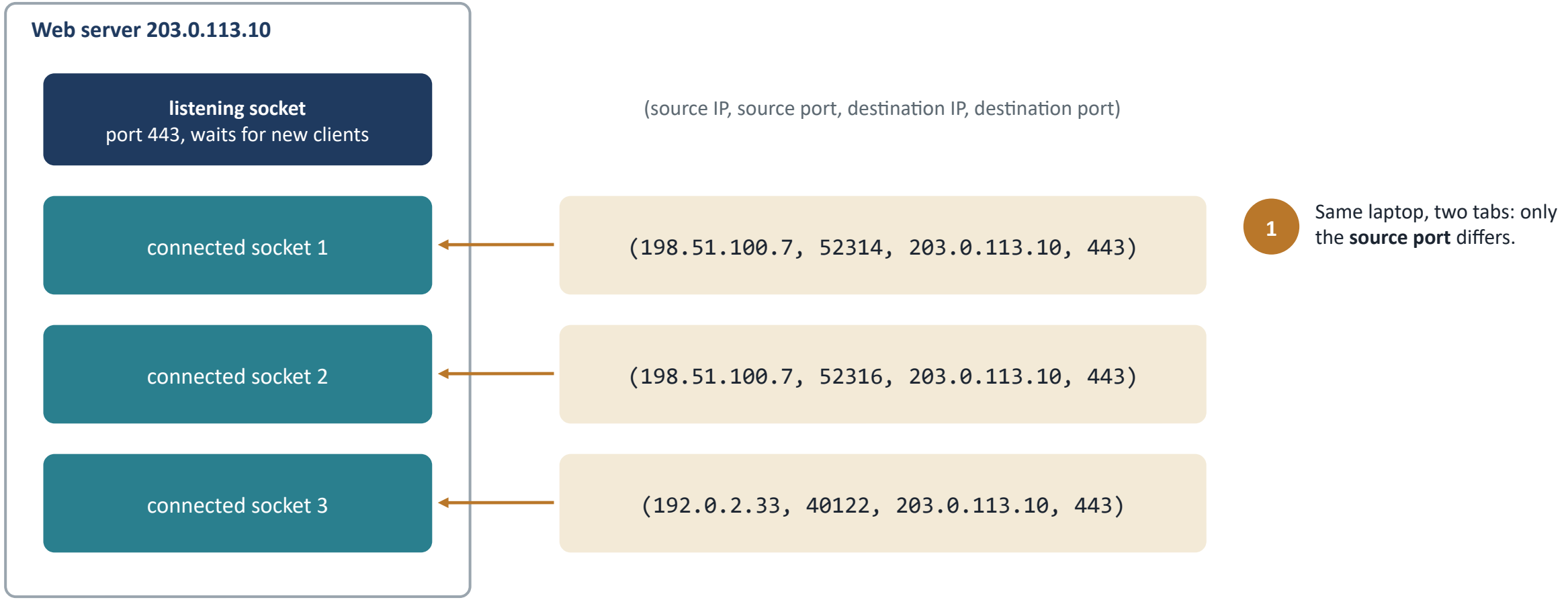
Well-known ports (servers)

80	HTTP (HyperText Transfer Protocol)
443	HTTPS (HyperText Transfer Protocol Secure)
53	DNS (Domain Name System)
25	SMTP (Simple Mail Transfer Protocol)
22	SSH (Secure Shell)

1 A client's socket gets a temporary **ephemeral port** chosen by its operating system (e.g. 51234).

Key idea IP address = which machine; port = which program on it. Servers use well-known ports, clients temporary ones.

3 · The 4-tuple tells connections apart



1 Same laptop, two tabs: only the **source port** differs.

Key idea A TCP (Transmission Control Protocol) connection is named by its 4-tuple; no two connections share all four numbers.

IP (Internet Protocol)

4 · What applications need from transport

1 Reliable delivery?

Must every byte arrive, intact and in order?

file transfer: yes · video call: some loss is fine

2 Throughput?

A minimum rate, or whatever is available (elastic)?

video stream: 5 Mbit/s · e-mail: elastic

3 Timing?

Must data arrive within a deadline?

games, voice: tens of ms · download: no

4 Security?

Confidentiality, integrity, authentication?

banking: essential

Key idea The answers to these four questions choose the transport protocol — not habit.

4 · Requirements of common applications

Application	Data loss	Throughput	Time-sensitive
File transfer	no loss	elastic	no
E-mail	no loss	elastic	no
Web pages	no loss	elastic	seconds
Voice and video calls	tolerant	kbit/s – Mbit/s	≈ 100 ms
Streaming stored video	tolerant	Mbit/s	a few s
Interactive games	tolerant	kbit/s and up	tens of ms
Text messaging	no loss	elastic	yes and no

Key idea Files and e-mail need every byte but not speed; calls and games need speed but tolerate loss.

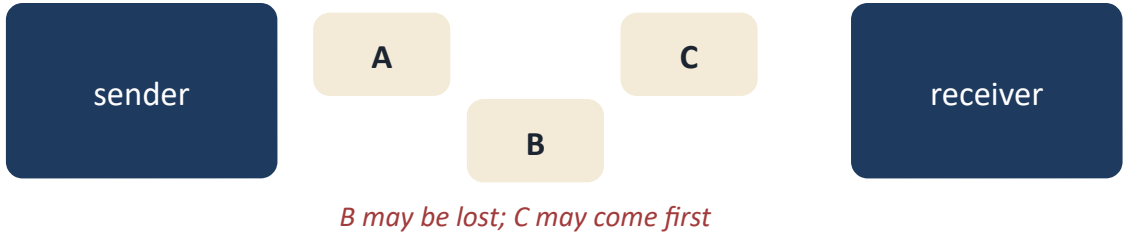
4 · What the Internet offers: TCP and UDP

TCP (Transmission Control Protocol)



- **connection** first (SYN (synchronize) / ACK (acknowledgement) handshake)
- **every byte** arrives, **in order**
- flow control and congestion control
- **no message boundaries** — one stream

UDP (User Datagram Protocol)



B may be lost; C may come first

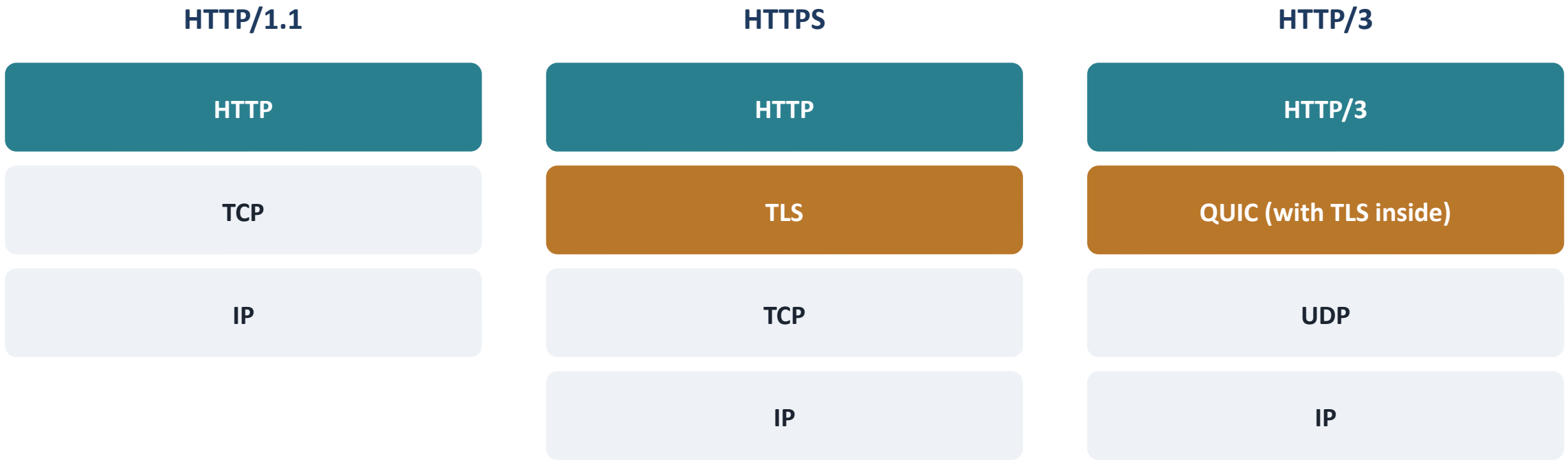
- **no** connection: just send
- datagrams may be **lost, duplicated, reordered**
- no flow or congestion control
- **boundaries kept**: one send = one datagram

Neither gives timing or throughput guarantees, and neither encrypts. TCP (Transmission Control Protocol): HTTP/1.1 and HTTP/2, SMTP, SSH. UDP (User Datagram Protocol): DNS, voice, video, games, QUIC.

Key idea TCP = reliable ordered pipe of bytes; UDP = independent envelopes with no promises.

HTTP (HyperText Transfer Protocol) · QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · SMTP (Simple Mail Transfer Protocol) · DNS (Domain Name System) · SSH (Secure Shell)

4 · Where TLS and QUIC sit



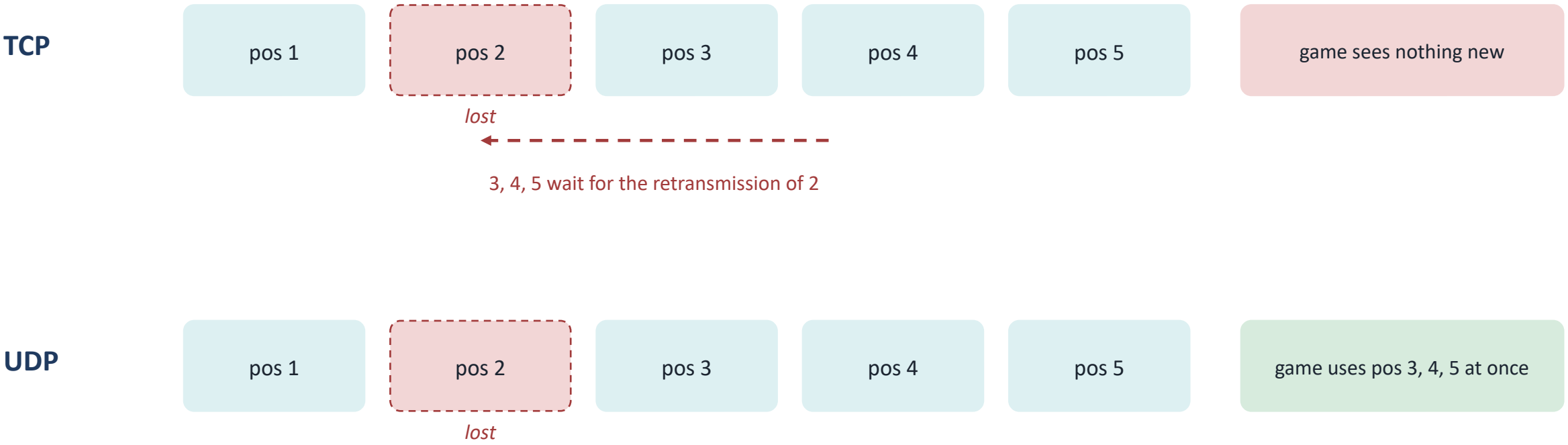
TLS (Transport Layer Security): a library the application calls instead of the plain socket — encryption and authentication over TCP (Transmission Control Protocol). HTTPS (HyperText Transfer Protocol Secure) = HTTP over TLS.

QUIC (a transport protocol over UDP; the name first stood for Quick UDP Internet Connections): under HTTP/3; HTTP (HyperText Transfer Protocol); UDP (User Datagram Protocol); IP (Internet Protocol).

Key idea Encryption is a layer you add (TLS) — or build into a new transport (QUIC).

QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections)

4 · Why would anyone choose UDP?



TCP (Transmission Control Protocol) also costs a round trip for its handshake and may slow a sender down. On UDP (User Datagram Protocol) an application adds exactly the reliability it needs — or builds a new transport, as QUIC does.

Key idea For a racing game, a late position is worse than a lost one — so it uses UDP.

QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections)

5 · What an application-layer protocol defines

```
GET /index.html HTTP/1.1
Host: www.example.org

HTTP/1.1 200 OK
Content-Length: 5120
…
```

1. **message types** — request · response
2. **syntax** — fields and how they are delimited
3. **semantics** — what each field means (200 = success)
4. **rules** — the client speaks first; one response per request

Open protocols: in public RFC (Request for Comments)s — HTTP, SMTP, DNS.
Proprietary: many messaging and call apps.

Key idea Types, syntax, semantics, rules: a checklist for reading — or designing — any protocol.

HTTP (HyperText Transfer Protocol) · SMTP (Simple Mail Transfer Protocol) · DNS (Domain Name System)

5 · An application is more than its protocol



The Web application also needs:

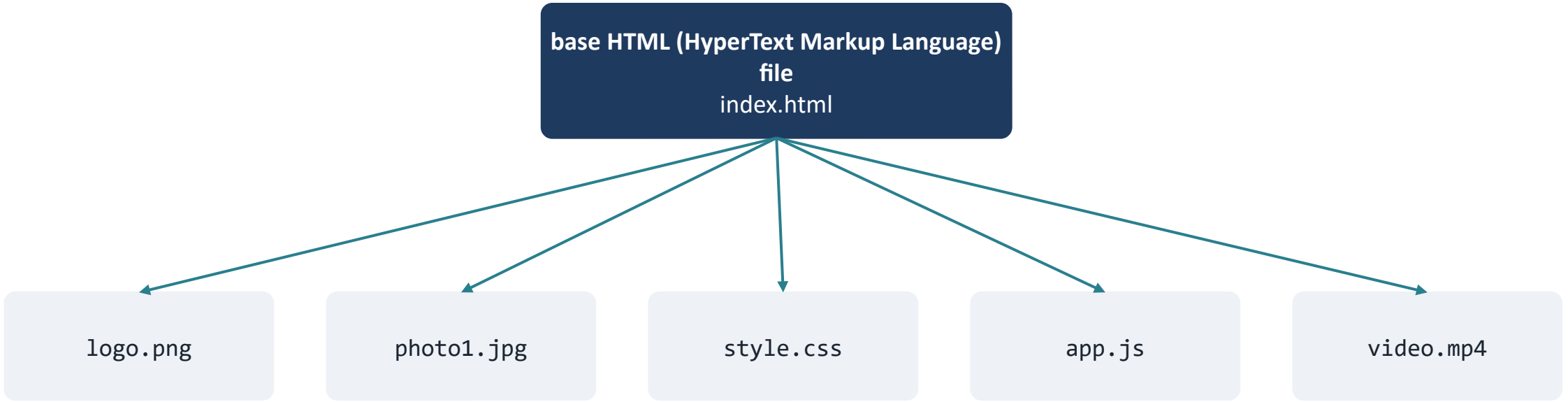
documents
HTML (HyperText Markup Language) files

standards
HTML, CSS (Cascading Style Sheets), HTTP

addresses
URL (Uniform Resource Locator)s

Key idea The Web = browsers + servers + documents + standards; HTTP is the protocol that links them.

6 · A web page is a tree of objects



1 Each **object** is a file with its own address and its own request.

2 A news page today references **well over a hundred** objects.

Key idea Loading one page means fetching one base file and then many objects — so round trips add up.

6 · Anatomy of a URL

URL (Uniform Resource Locator)



scheme
which protocol: http, https

host
resolved by DNS (Domain Name System) to an IP address

port
optional: 80 for http, 443 for https by default

path
which object on the server

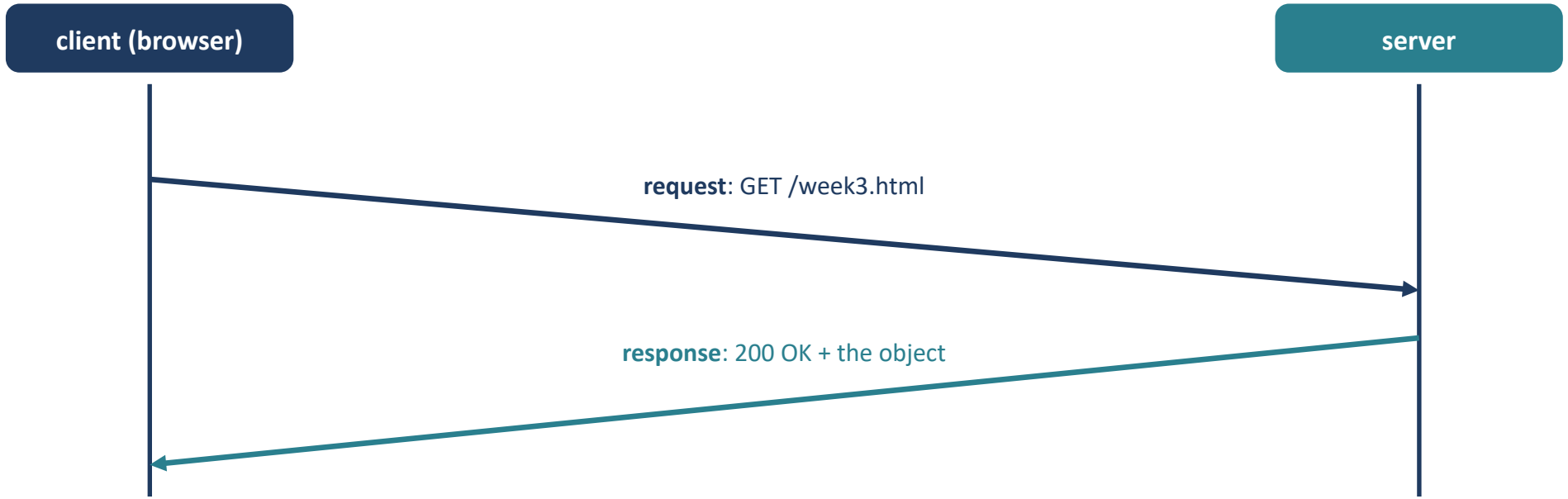
query
parameters for the server

fragment
used only by the browser — never sent

Key idea scheme + host + port find the server; path + query name the object; the fragment stays in the browser.

IP (Internet Protocol)

6 · HTTP: request and response



1

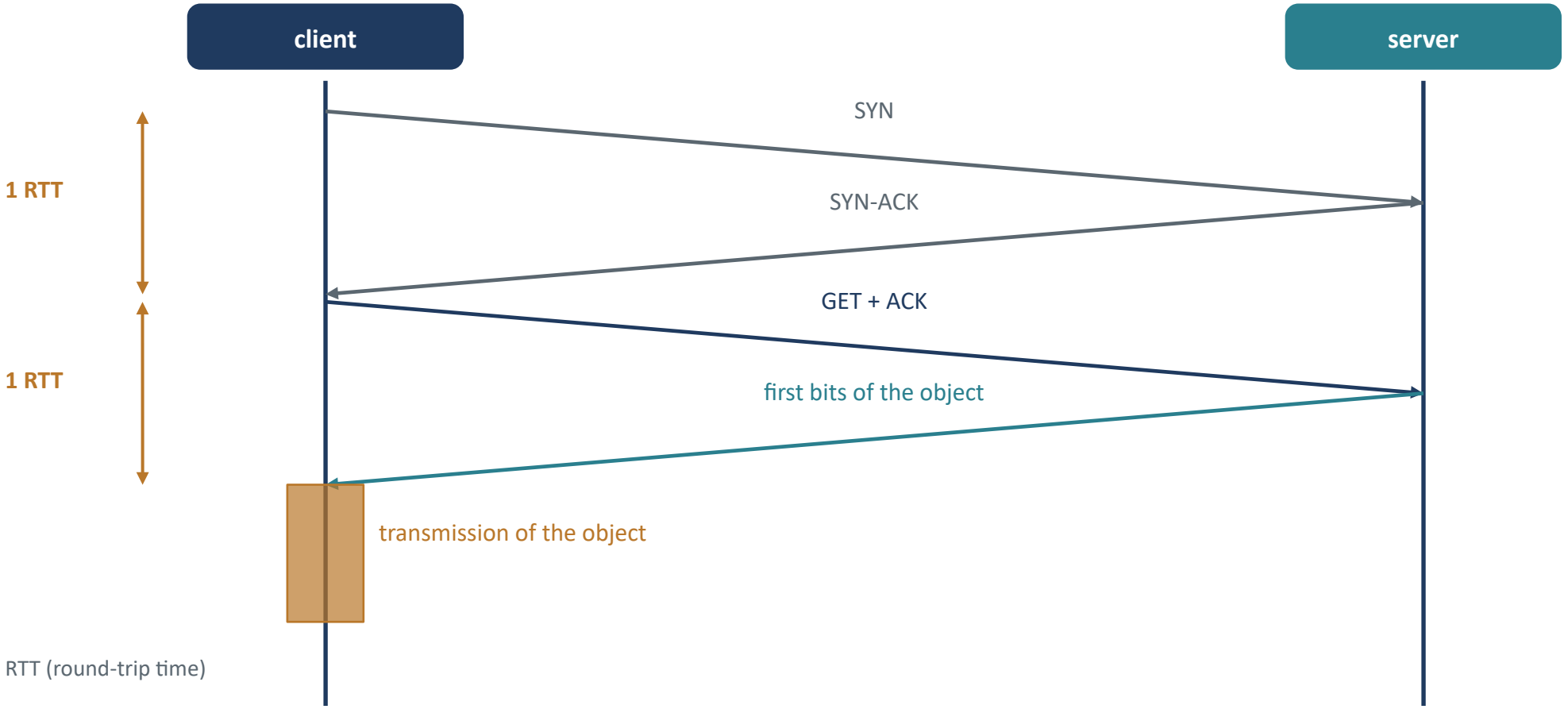
Runs over TCP (Transmission Control Protocol) (HTTP/1.1, HTTP/2) or over QUIC (HTTP/3). HTTP (HyperText Transfer Protocol). State (logins, baskets) is added with **cookies** — next week.

stateless
no memory of earlier requests

Key idea HTTP is a stateless request–response protocol: simpler, more robust servers.

QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections)

7 · One object, non-persistent: 2 RTT + transmission

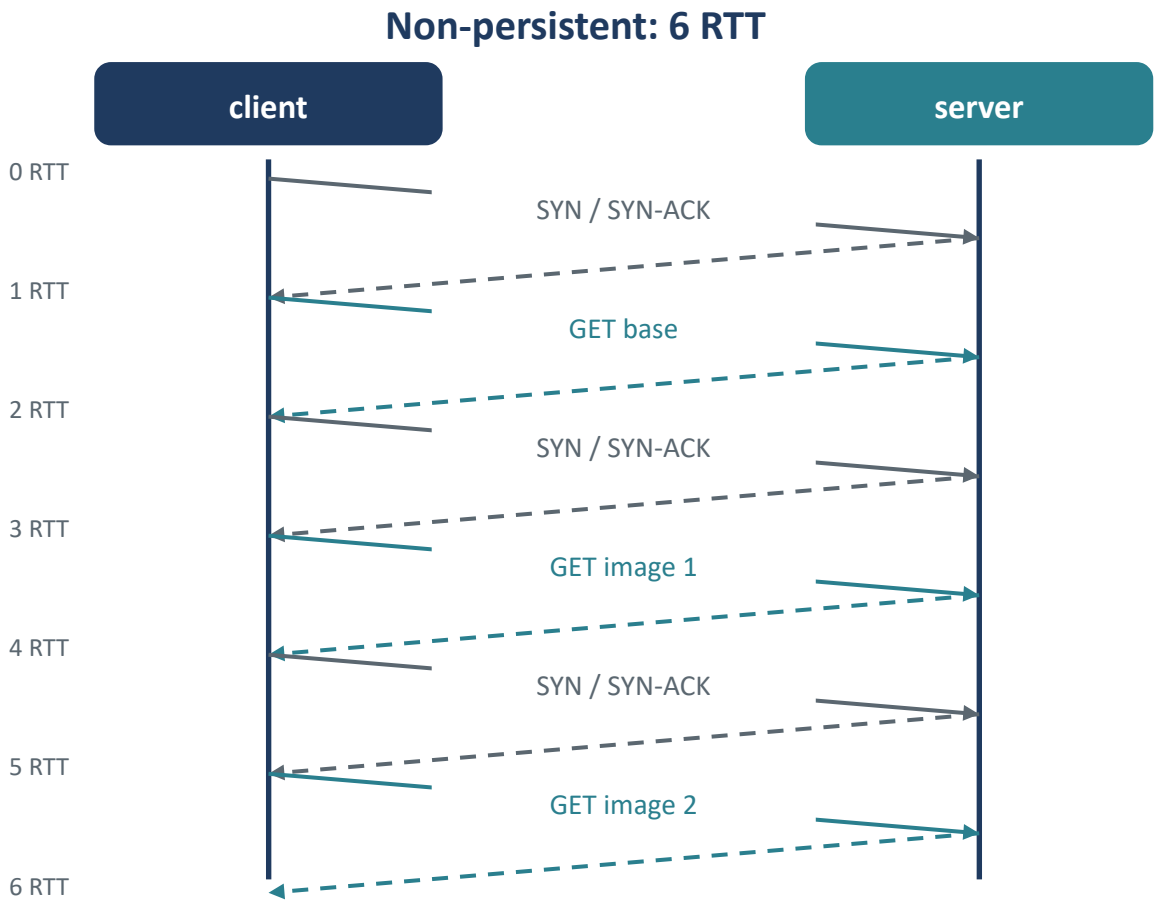


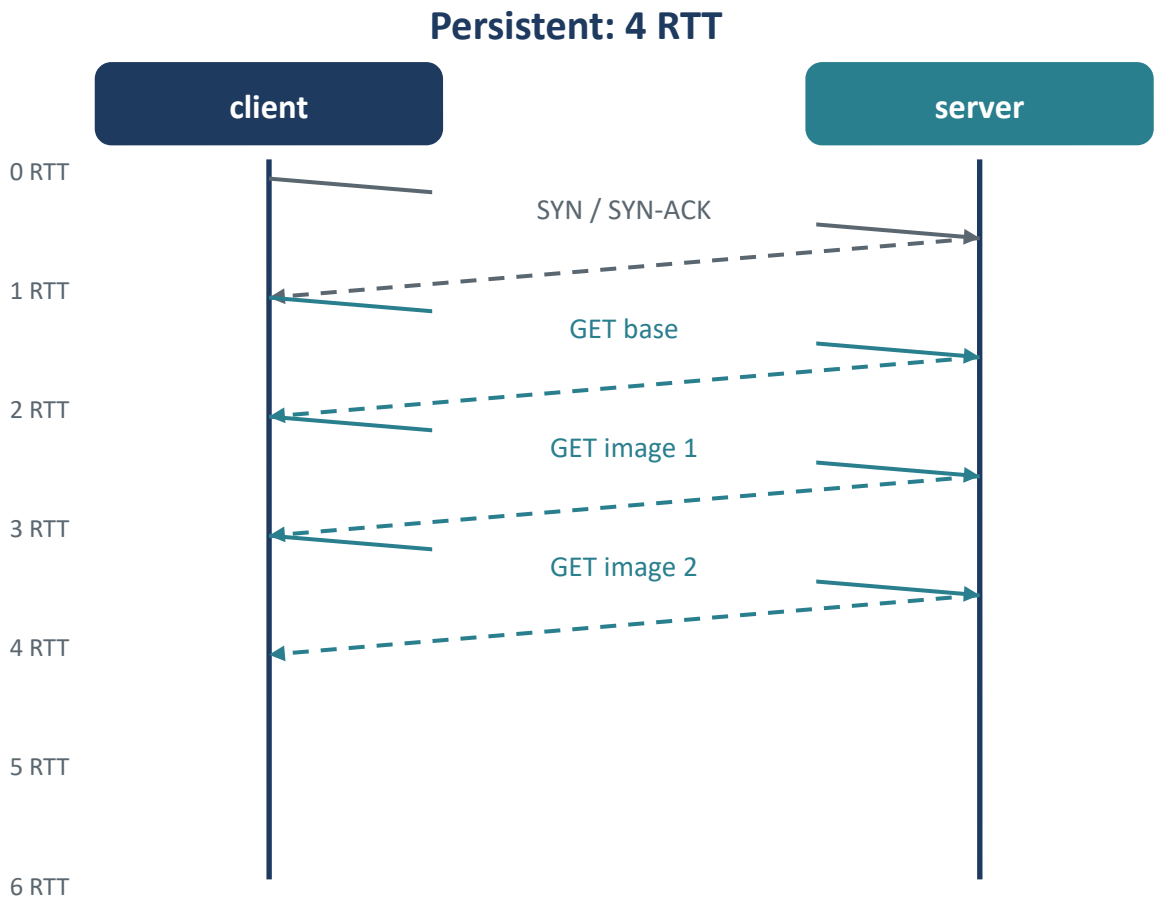
time per object
2 RTT
+ transmission

Key idea Non-persistent HTTP pays a TCP (Transmission Control Protocol) handshake for every object: 2 RTT each.

HTTP (HyperText Transfer Protocol) · SYN (synchronize) · ACK (acknowledgement)

7 · Non-persistent vs persistent: base page + 2 images



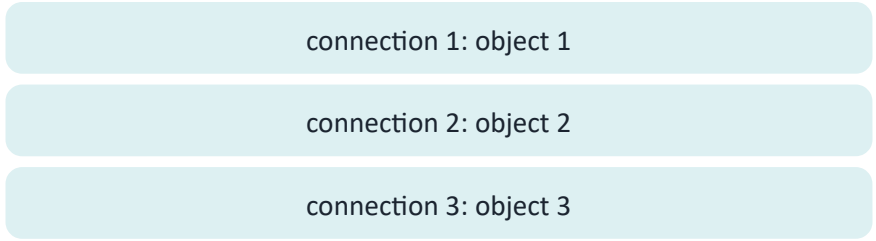


Key idea Persistent connections (the HTTP/1.1 default) pay the handshake once: 1 RTT per object after the first.

HTTP (HyperText Transfer Protocol) · RTT (round-trip time) · SYN (synchronize) · ACK (acknowledgement)

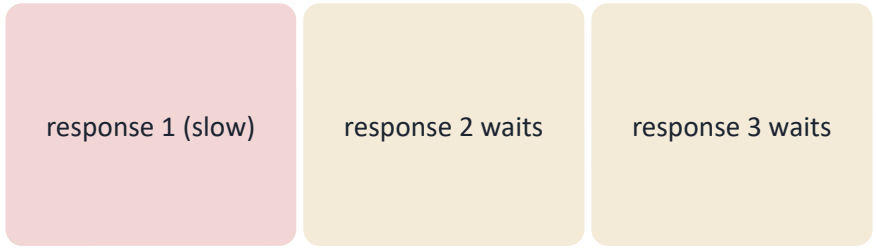
7 · Doing several things at once

Parallel connections
several TCP connections at once (≈ 6 per server in HTTP/1.1)



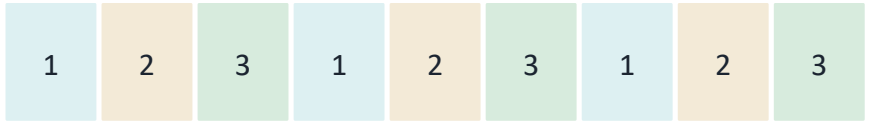
hides latency · more server load · unfair to others

Pipelining (HTTP/1.1)
several requests on one connection; responses **in order**



one slow response blocks the rest: **head-of-line blocking**

Multiplexing (HTTP/2, HTTP/3)
requests split into frames, **interleaved** on one connection



no head-of-line blocking at the HTTP level

Key idea Parallelism hides round trips; HTTP/2 and HTTP/3 do it on one connection without head-of-line blocking.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)

7 · Worked example: 1 base page + 10 images, RTT = 50 ms

Non-persistent, one at a time

11 objects × 2 RTT = 22 RTT



1100 ms

Non-persistent, 5 parallel

2 RTT + 2 rounds × 2 RTT = 6 RTT



300 ms

Persistent, one request at a time

2 RTT + 10 × 1 RTT = 12 RTT



600 ms

Persistent + pipelining / HTTP/2

2 RTT + 1 RTT = 3 RTT



150 ms

Key idea For pages of many small objects, the number of round trips — not the link rate — sets the load time.

HTTP (HyperText Transfer Protocol) · RTT (round-trip time)

8 · Anatomy of an HTTP/1.1 request

```
GET /courses/networks/week3.html HTTP/1.1
Host: www.example.org
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: text/html
Connection: keep-alive
(empty line)
```

1. **Request line:** method · target (path and query) · version.
2. **Host** is mandatory: one IP address may host thousands of sites.
3. **Header lines**: `name: value`, one per line.
4. **Empty line** (CR LF (carriage return, line feed) alone) ends the header; a **body** may follow (form data).

Specified in RFC (Request for Comments) 9110 (HTTP semantics) and RFC 9112 (HTTP/1.1 messages), 2022.

Key idea A request is lines of text: request line, headers, empty line, optional body.

HTTP (HyperText Transfer Protocol) · X11 (X Window System, version 11) · IP (Internet Protocol)

8 · Methods: safe and idempotent

safe	**safe + idempotent** **GET** retrieve **HEAD** header only **OPTIONS** which methods?	**safe, not idempotent** *(no standard method)*
not safe	**idempotent, not safe** **PUT** create or replace **DELETE** delete	**neither** **POST** submit, e.g. an order **PATCH** modify part

Key idea A client may repeat GET, PUT or DELETE after a lost response; repeating POST may order twice.

8 · Anatomy of an HTTP/1.1 response

```
HTTP/1.1 200 OK
Last-Modified: Fri, 16 Oct 2026 14:12:30 GMT
Content-Type: text/html
Content-Length: 5120
(empty line)
<!doctype html> … (5120 bytes of body)
```

1. **Status line:** version · status code · reason phrase.
2. **Headers** describe the body (type, size, date). GMT = Greenwich Mean Time.
3. **Content-Length** says exactly how many body bytes follow.
4. **Body**: the object itself, after the empty line.

Key idea The response mirrors the request: a first line, headers, empty line, then exactly Content-Length bytes.

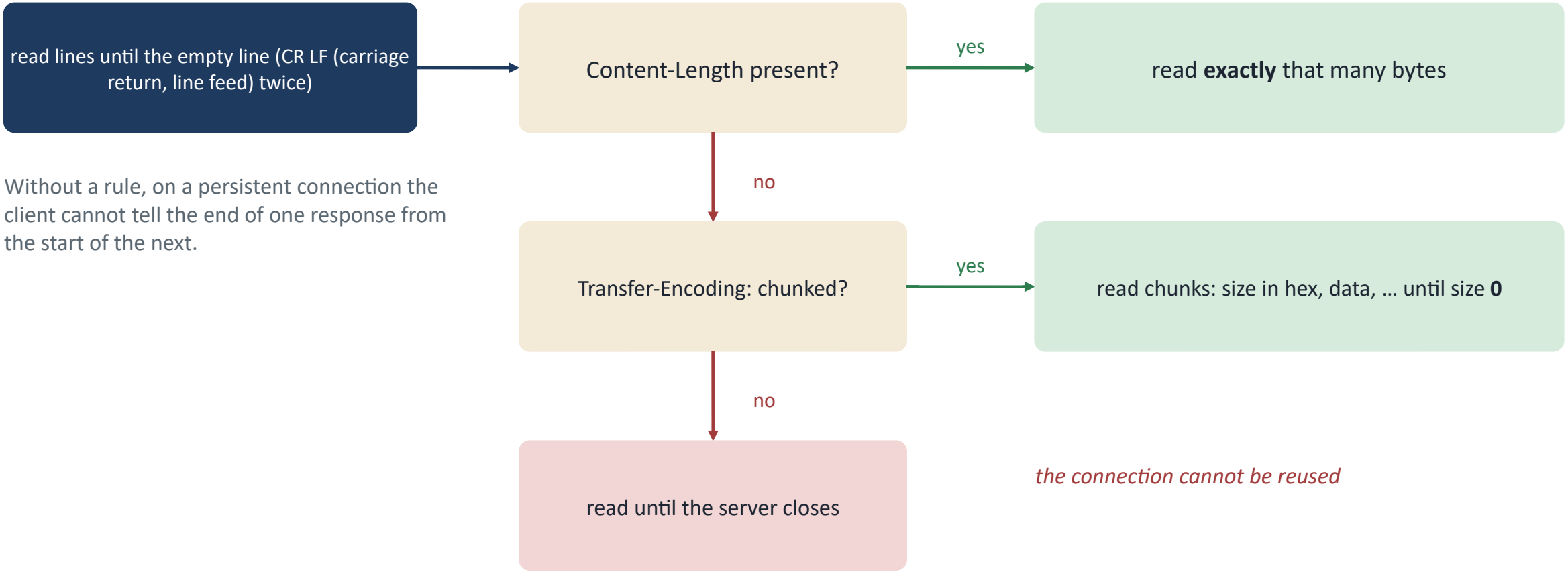
HTTP (HyperText Transfer Protocol)

8 · Status codes: five classes

1xx	**informational**	100 Continue · 101 Switching Protocols
2xx	**success**	200 OK · 201 Created · 204 No Content
3xx	**redirection: look elsewhere**	301 Moved Permanently · 302 Found · 304 Not Modified
4xx	**client error: fix the request**	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 405 Method Not Allowed
5xx	**server error: not your fault**	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

Key idea 2xx: done · 3xx: go elsewhere · 4xx: the client erred · 5xx: the server failed.

8 · Where does a message end? The framing rules

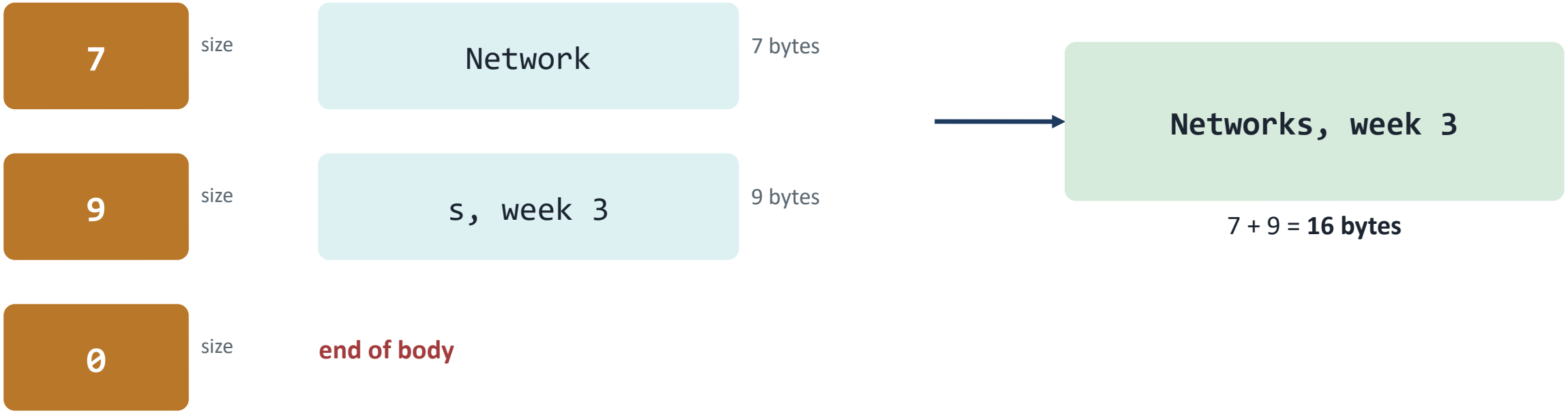


Without a rule, on a persistent connection the client cannot tell the end of one response from the start of the next.

Key idea Every protocol over TCP must solve framing; HTTP/1.1 uses an empty line, a length, or chunks.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)

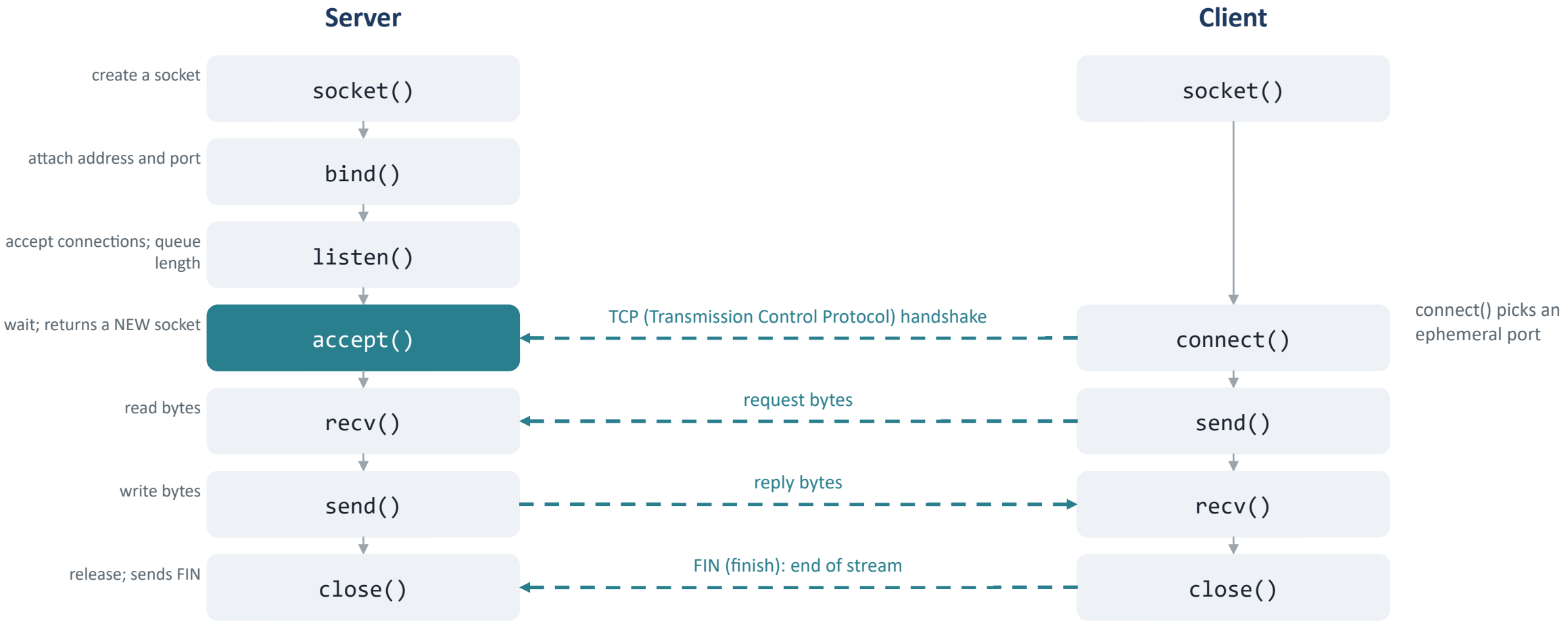
8 · Chunked encoding, decoded



1 Each size and each piece of data is followed by CR LF (carriage return, line feed). Used when the server does not know the length in advance (a page generated while it is sent).

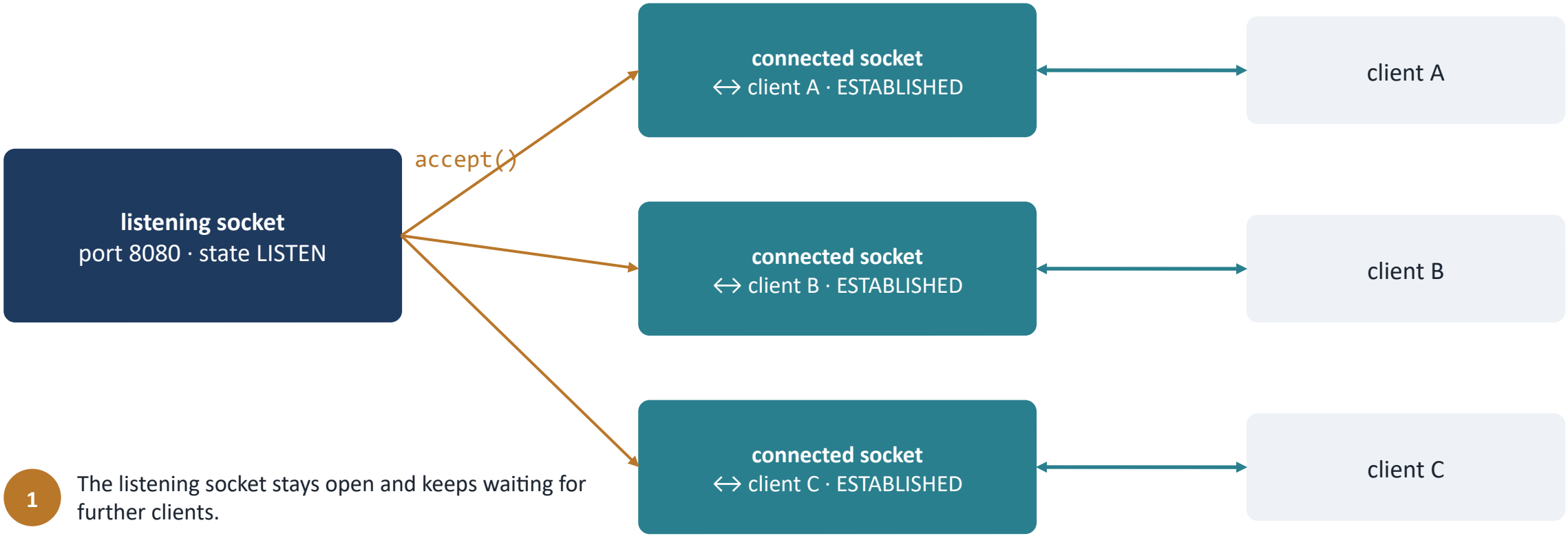
Key idea Chunked encoding lets a server stream a body of unknown length and still keep the connection.

9 · The socket calls of a TCP server and client



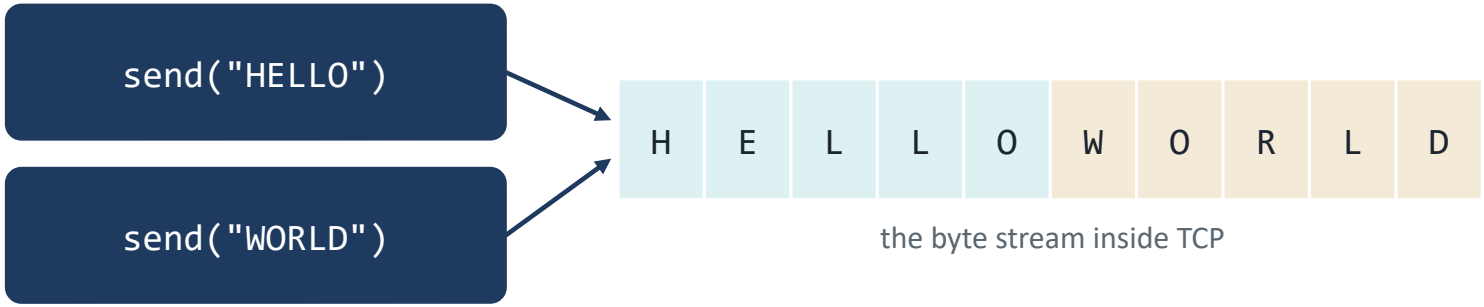
Key idea Server: socket → bind → listen → accept. Client: socket → connect. Both: send, recv, close.

9 · accept() returns a new socket



Key idea One listening socket per service; one connected socket per client.

9 · A stream has no message boundaries



1 Never possible: lost or **reordered** bytes.

possible: recv → 10

HELLOWORLD

2 So a program must **loop** until it has what it expects, and the protocol must say where a message ends.

possible: recv → 5, recv → 5

HELLO WORLD

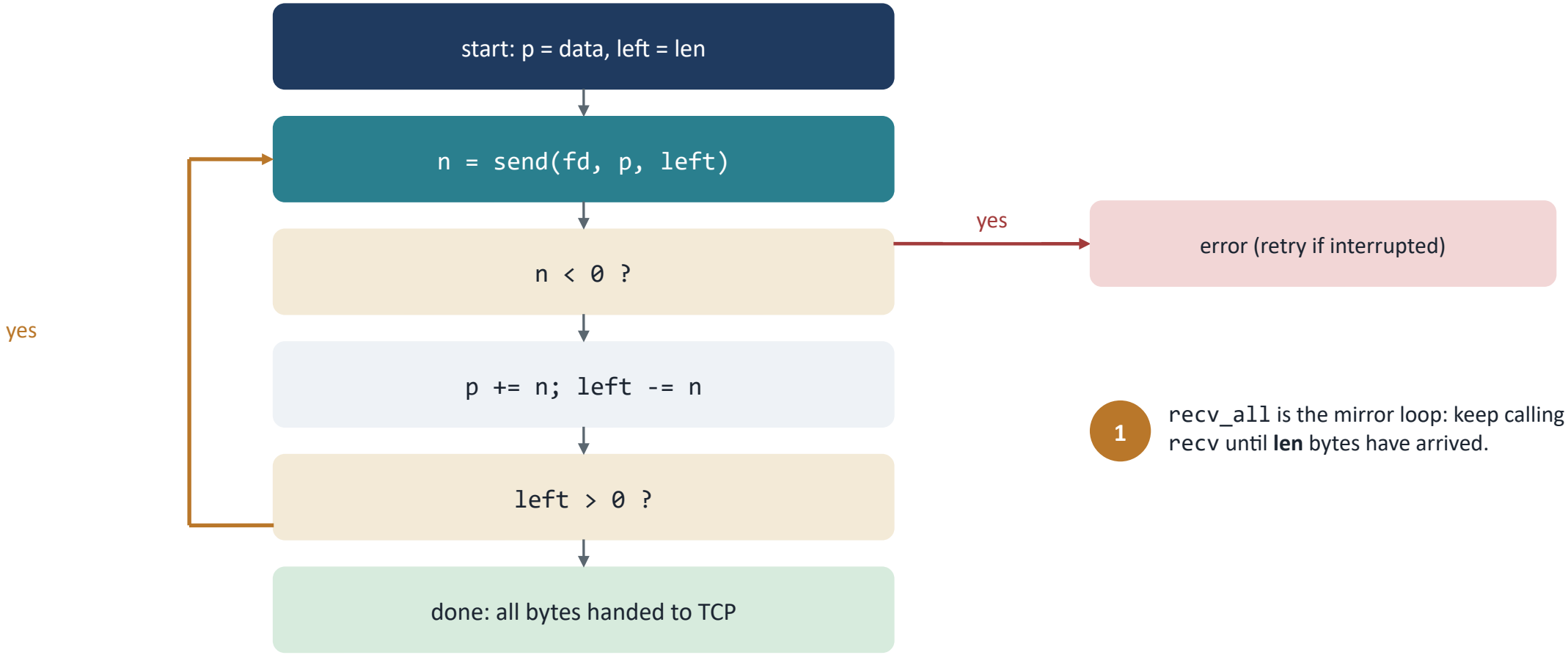
possible: recv → 3, recv → 7

HEL LOWORLD

Key idea TCP preserves bytes and their order — not the boundaries between your sends.

TCP (Transmission Control Protocol)

9 · send() may accept fewer bytes: the send_all loop



1 `recv_all` is the mirror loop: keep calling `recv` until **len** bytes have arrived.

Key idea Always wrap send in a loop (send_all; sendall in Python) and recv in a loop when you need an exact count.

TCP (Transmission Control Protocol) · fd (file descriptor)

9 · Byte order: htons(8080)

8080 = **0x1F90**

memory of an x86 machine (little-endian)

90	1F



htons() (host to network short)

network byte order (big-endian)

1F	90

Forget htons:
the bytes 90 1F go out as they are
and are read as **0x901F = 36 895**
→ the server listens on port 36 895

The four converters
htons · htonl: host to network (short = 16 bits, long = 32 bits)
ntohs · ntohl: network to host

Key idea Convert every port, address and binary integer to network byte order before sending.

htonl (host to network long) · ntohl (network to host long) · ntohs (network to host short)

Lecture 3 summary

Architectures
client–server, peer-to-peer, hybrid

Framing
empty line, Content-Length, chunked; socket calls

Sockets
process = IP address + port; connection = 4-tuple

Round trips
2 RTT per object non-persistent, 1 RTT persistent

Week 3

Transport needs
reliability, throughput, timing, security

The Web
objects, URLs, stateless HTTP

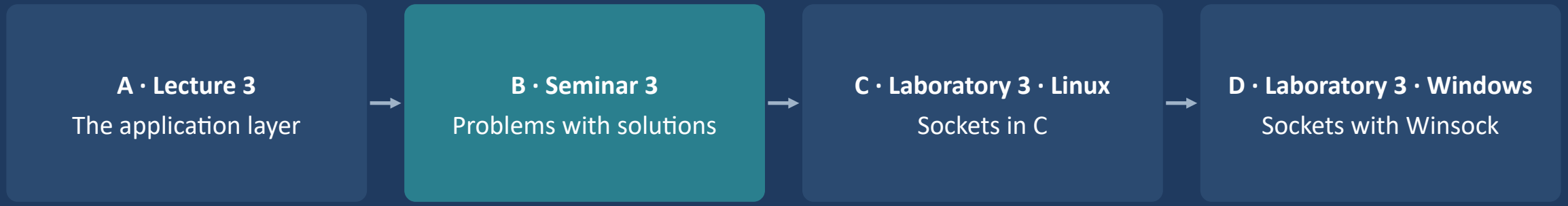
TCP vs UDP
reliable stream vs bare datagrams; TLS adds security

Protocol
types, syntax, semantics, rules

Key idea From architectures to bytes on a socket: everything an application programmer needs this week.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · RTT (round-trip time) · IP (Internet Protocol)

Part B · Seminar 3: problems with solutions



In this part

Problem 1 · Which transport?

Problem 4 · Reading an HTTP exchange

Problems 7–8 · Status codes, retries

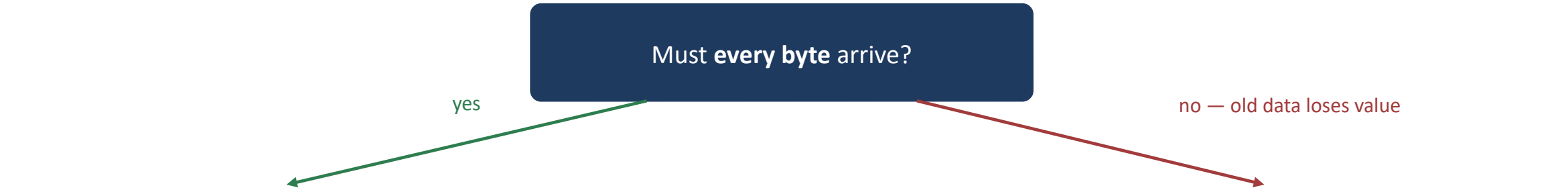
Problems 2–3 · Page-load times

Problems 5–6 · Framing

Problems 9–10 · Sockets and a design

HTTP (HyperText Transfer Protocol)

Problem 1 · Choosing a transport: a decision tree



TCP (Transmission Control Protocol)

a
remote login (SSH, Secure Shell)

c
2 GB backup upload

UDP (User Datagram Protocol)

b
racing game, 30 positions/s

d
video call → RTP (Real-time Transport Protocol) on top

e
DNS (Domain Name System) query: client retries itself

f
sensor reading every minute

1 If every reading of (f) matters: add acknowledgements and retries on top of UDP — or use TCP.

Key idea Loss-intolerant and elastic → TCP. Time-sensitive and loss-tolerant → UDP, plus only what the application needs.

Problem 2 · Page-load time: base page + 6 images

Base page 50 KB · 6 images × 100 KB · RTT 40 ms · access link 20 Mbit/s → base 20 ms, each image 40 ms to transmit

Non-persistent, one at a time

base 2·40+20; 6 × (2·40+40)



820 ms

Persistent, no pipelining

base 100; 6 × (40+40)



580 ms

Persistent + pipelining

base 100; 40 + 6 × 40



380 ms

RTT (round-trip time)

round trips (RTT)

transmission

1 Image transmissions (240 ms) never shrink; only the navy round trips can be removed.

Key idea Persistence removes handshakes, pipelining removes waiting — transmission time stays.

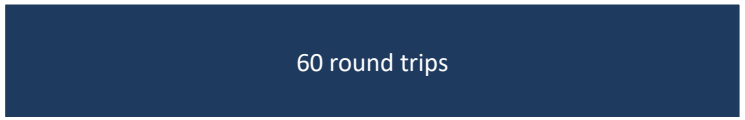
Problem 2.5 · The same page with 60 images of 2 KB

Non-persistent
60 × (80 + 0.8) ms



≈ 4.8 s

Persistent, no pipelining
60 × (40 + 0.8) ms



≈ 2.4 s

Pipelining
40 + 60 × 0.8 ms



88 ms

RTT (round-trip time)

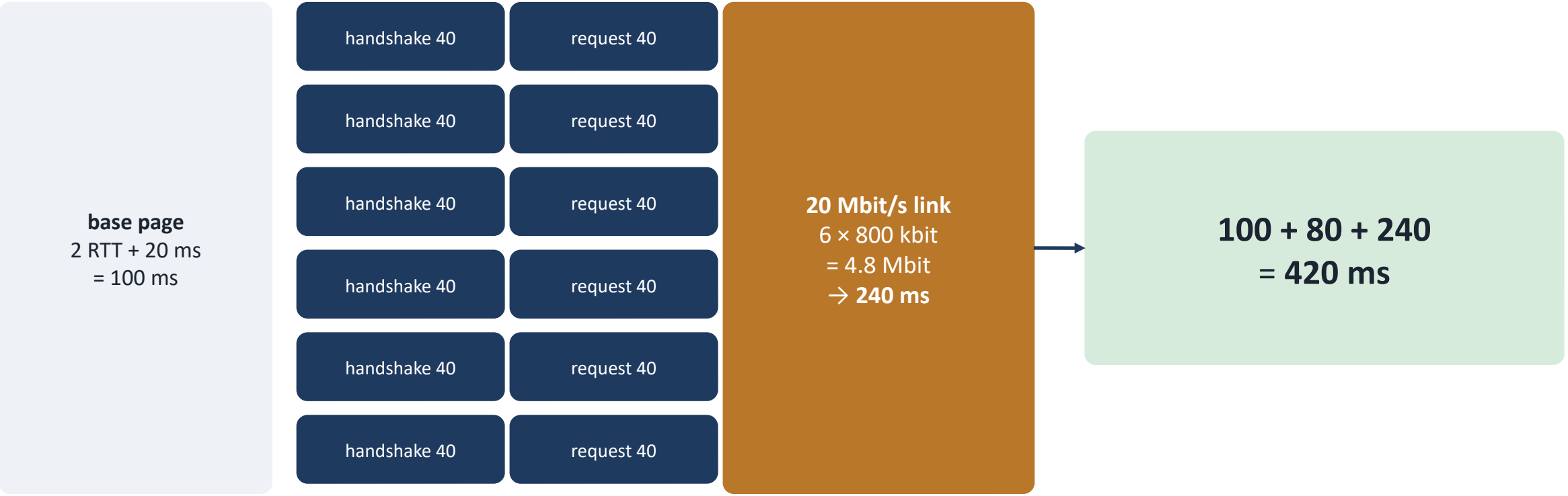
round trips (RTT)

transmission

1 Times after the base page. With many small objects the **number of round trips** decides everything.

Key idea Many small objects: count round trips, not bytes.

Problem 3 · Six parallel connections share one link

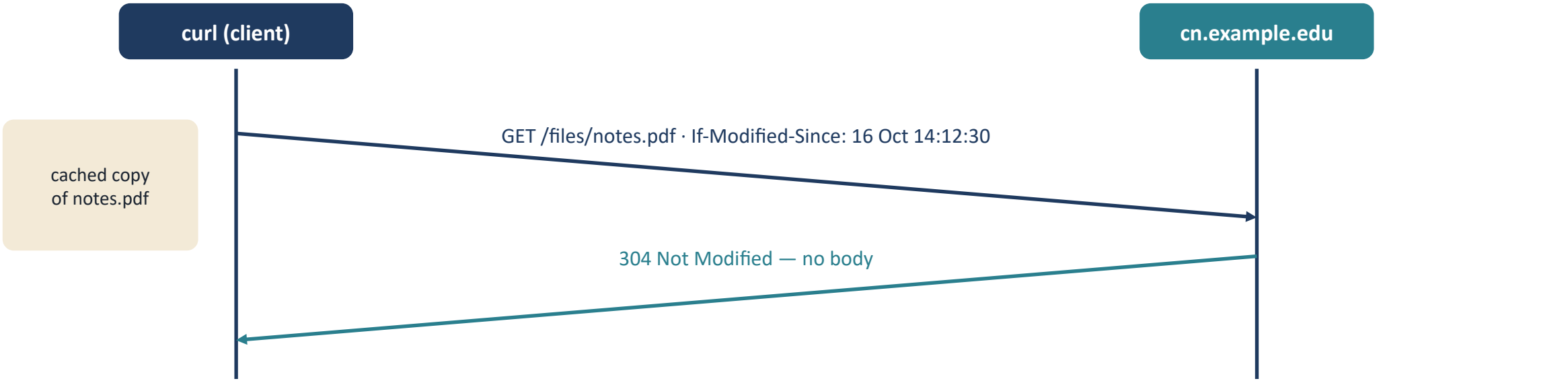


1 Not 6 times better than 820 ms: parallelism hides round trips, **not transmission**.

Key idea Parallel connections overlap waiting, but they share the same bottleneck link.

RTT (round-trip time)

Problem 4 · A conditional GET answered with 304



full URL
http://cn.example.edu/files/notes.pdf

persistent?
yes: HTTP/1.1, no Connection: close

body length
0 — a 304 never has a body

why
check freshness without downloading again

Key idea A conditional GET saves the transfer: 304 means “your copy is still good” (caching, next week).

HTTP (HyperText Transfer Protocol) · URL (Uniform Resource Locator)

Problem 5 · Decoding a chunked body

`1a`	= 26 bytes	`abcdefghijklmnopqrstuvwxyz`
`a`	= 10 bytes	`0123456789`
`0`	= 0 bytes	**end of body** (then an empty line)

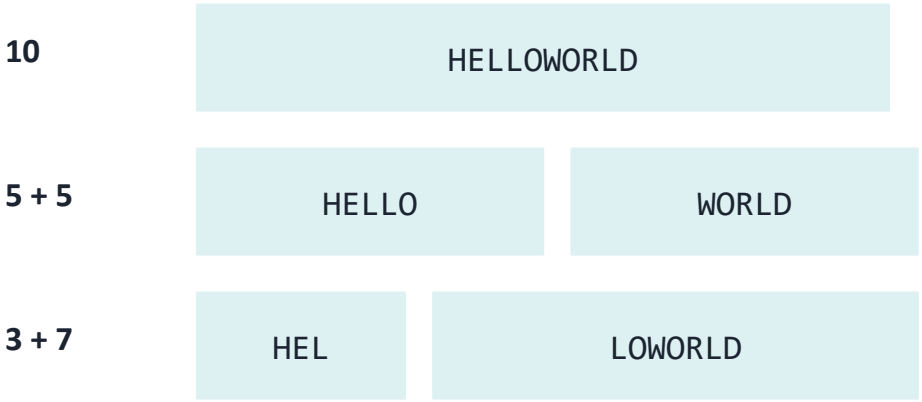
26 + 10
= 36 bytes

1. After the 0-size chunk the connection is free for the **next** response — framing makes reuse possible.

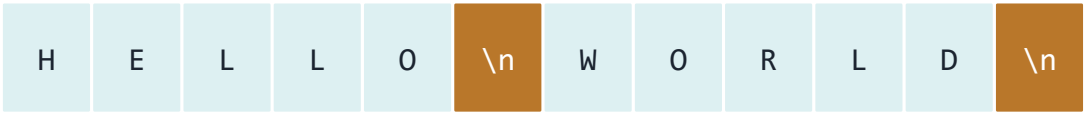
Key idea Read the hexadecimal size, then exactly that many bytes; repeat until size 0.

Problem 6 · Two sends, many possible receives — and two fixes

What the server may see



Fix 1 · delimiter (12 bytes)



read until each \n — the delimiter must never appear inside a message

Fix 2 · length prefix (18 bytes) — works for any content, even binary



4-byte length in network byte order, then exactly that many bytes

Key idea Bytes always arrive in order — boundaries never do. Mark them with a delimiter or a length.

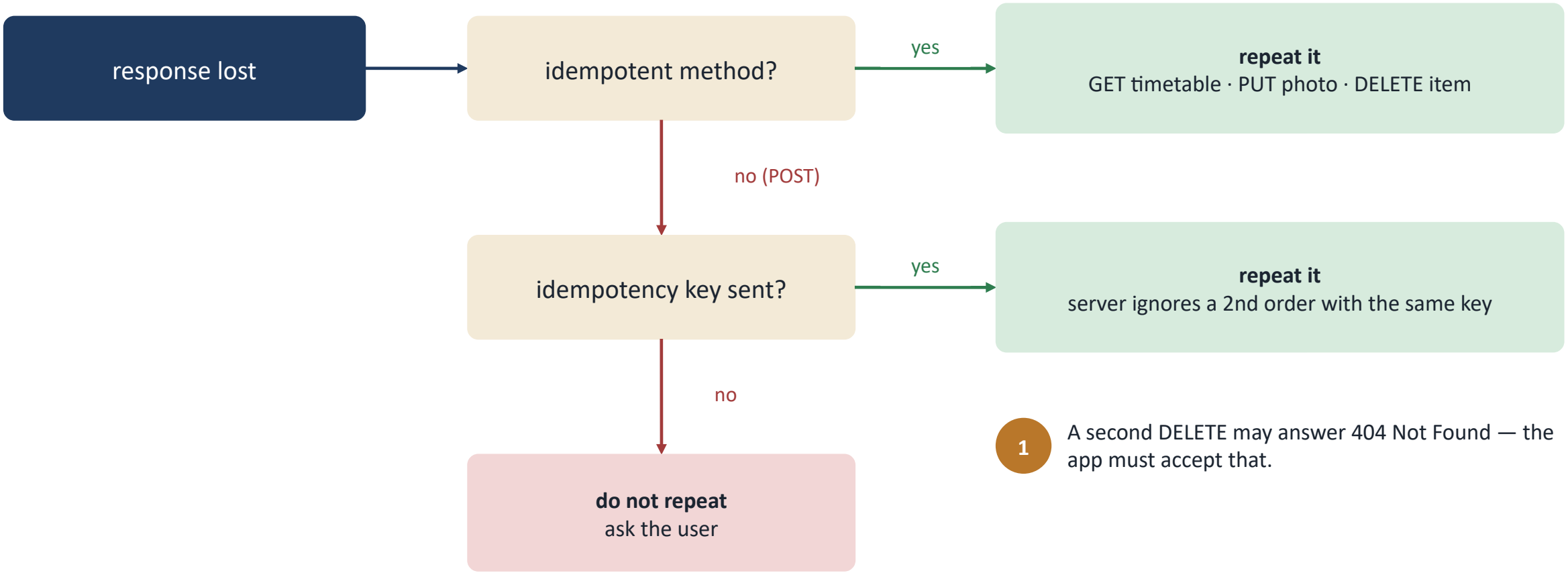
Problem 7 · Matching situations to status codes

site moved for good to https://	301 Moved Permanently
/admin without logging in	401 Unauthorized
/admin as a student (logged in)	403 Forbidden
DELETE on a server that never deletes	405 Method Not Allowed
POST created a new account	201 Created
application crashed	500 Internal Server Error
request line “GTE / HTTP/1.1”	400 Bad Request

Key idea 401 = who are you? · 403 = I know you, but no · 405 = not with this method.

HTTP (HyperText Transfer Protocol)

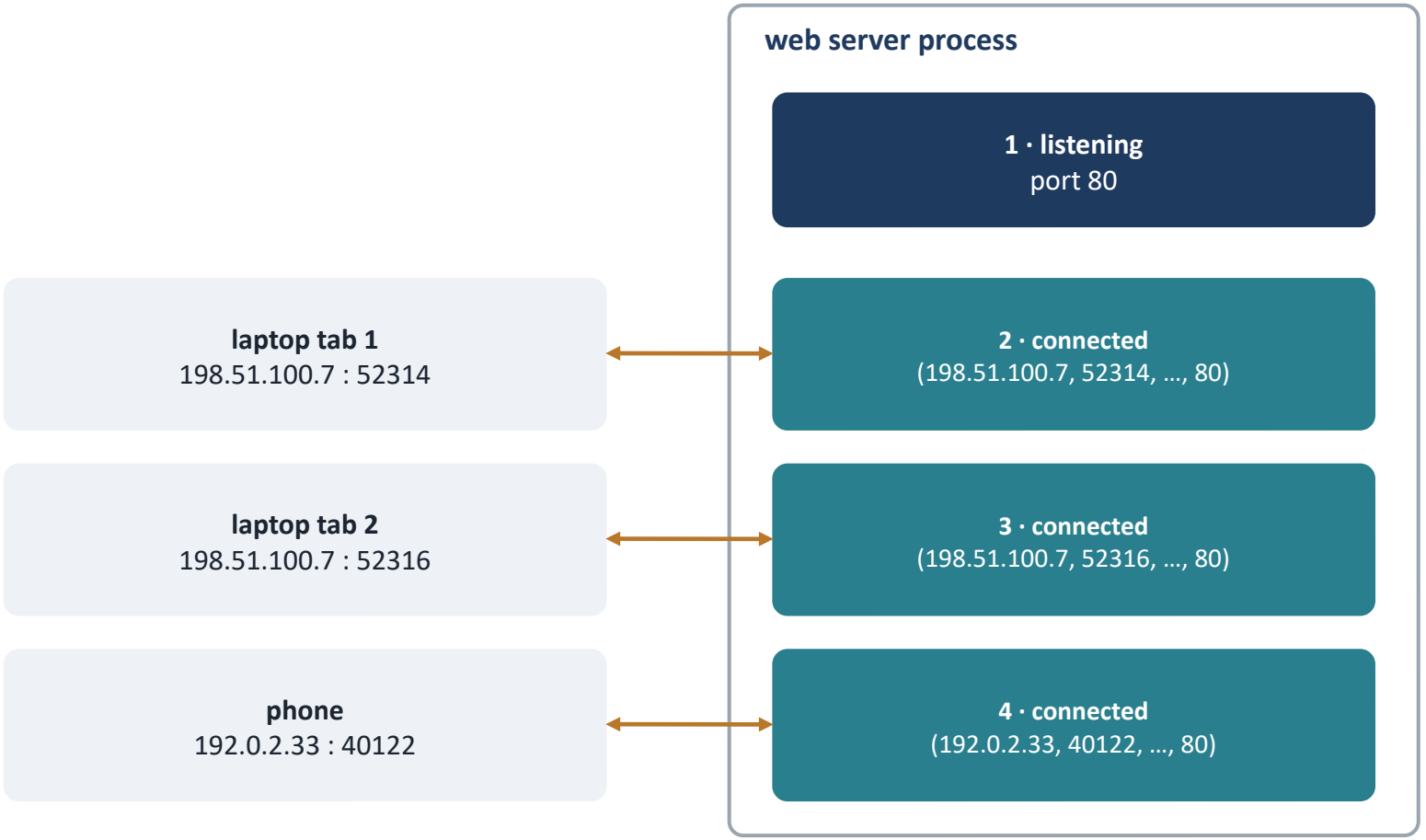
Problem 8 · May the app repeat a request after a lost response?



1 A second DELETE may answer 404 Not Found — the app must accept that.

Key idea Repeat idempotent requests freely; repeat a POST only with a key the server can recognise.

Problem 9 · Sockets and 4-tuples on a busy server



1 The two laptop tabs differ **only in the source port** — enough to tell them apart.

Key idea Sockets on a server = 1 listening + 1 per connection; each connection has a unique 4-tuple.

Problem 10 · Designing a file-upload protocol

Request

```
PUT notes.pdf 52113\r\n
```

header line ends at the first CR LF (carriage return, line feed)

52 113 raw bytes of the file

body ends after exactly <size> bytes

Response

```
OK\r\n
```

```
ERR 413 too large\r\n
```

names
no spaces, no CR LF

several requests
may follow on one connection

repeat
PUT of the same file is idempotent

early close
fewer than <size> bytes → discard

Key idea Delimiter for the text header (safe: names cannot contain it), length for the binary body.

Part C · Laboratory 3 on Linux: sockets in C

A · Lecture 3
The application layer

→

B · Seminar 3
Problems with solutions

→

C · Laboratory 3 · Linux
Sockets in C

→

D · Laboratory 3 · Windows
Sockets with Winsock

In this part

Setup · files, make, terminals

Task 1 · A server without code (nc)

Task 2 · The echo client

Task 3 · The echo server

Task 4 · Sockets seen by the OS

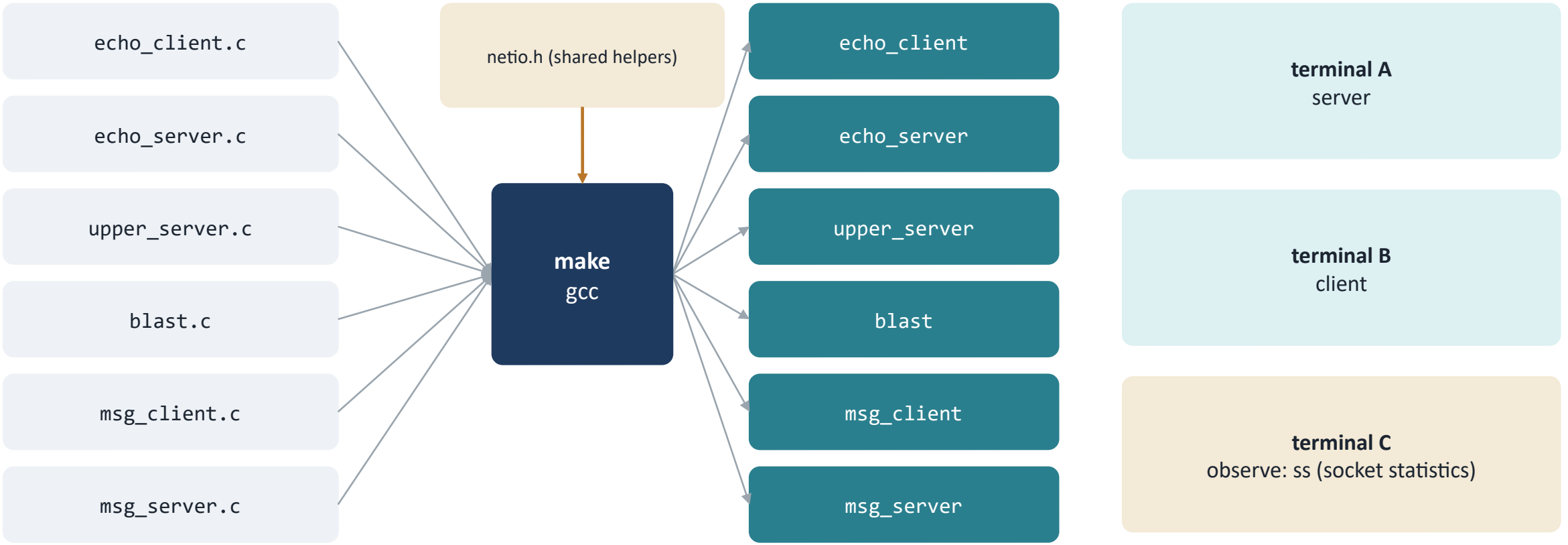
Task 5 · A stream is not messages

Tasks 6–7 · Two framing methods

Task 8 · Breaking things on purpose

OS (operating system)

Setup · From source files to running programs



Install once: build-essential, netcat-openbsd, manpages-dev · use ports 5000–60000

Key idea Build everything with make; run the server in one terminal, the client in another, and watch from a third.

Task 1 · A server without writing code: nc and ss



before the client

```
LISTEN  0.0.0.0:5000
```

after it connects

```
ESTAB  127.0.0.1:5000 ↔ 127.0.0.1:41822
```

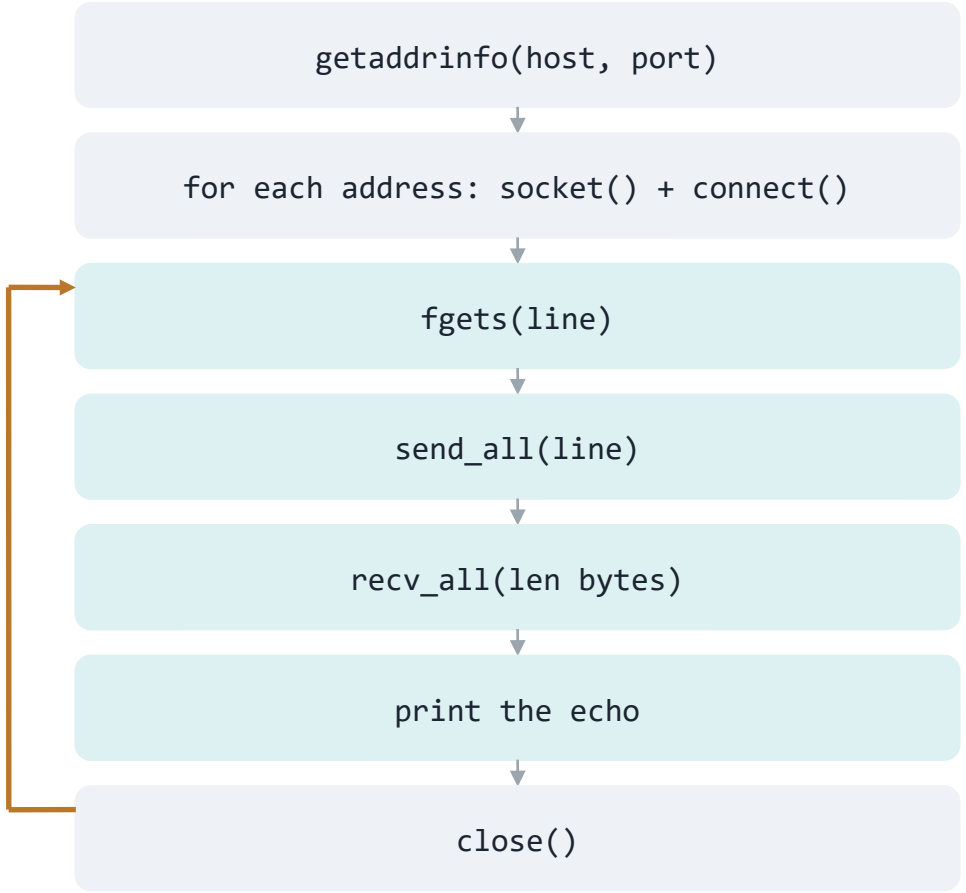
1. Q1 · port 41822 was chosen by the client's operating system: an **ephemeral port**.

2. Q2 · Ctrl+D in B sends FIN (finish); nc in A sees the end of the stream and exits.

Key idea A listening socket waits in LISTEN; each accepted connection appears as ESTAB with its 4-tuple.

ss (socket statistics)

Task 2 · The echo client, step by step



name → list of addresses (IPv6 and IPv4)

stop at the first that succeeds

read a line from the keyboard

write all its bytes

read exactly as many bytes back

then the next line

at end of input (Ctrl+D)

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6)

1 Q3 · one `recv` may return **part** of the echo; `recv_all` loops until all **len** bytes are in.

2 Q4 · nothing listening → "Connection refused": the host answered the SYN (synchronize) with RST (reset).

Key idea A robust client resolves names, tries every address, and reads exact byte counts.

Task 3 · The echo server: six steps

1. `socket()` — create
2. `bind(0.0.0.0:port)` — attach to all local addresses
3. `listen(8)` — queue up to 8 pending clients
4. `accept()` — wait; returns a NEW socket
5. `recv()` → `send_all()` — echo whatever arrived
6. `close(client)` — then back to step 4

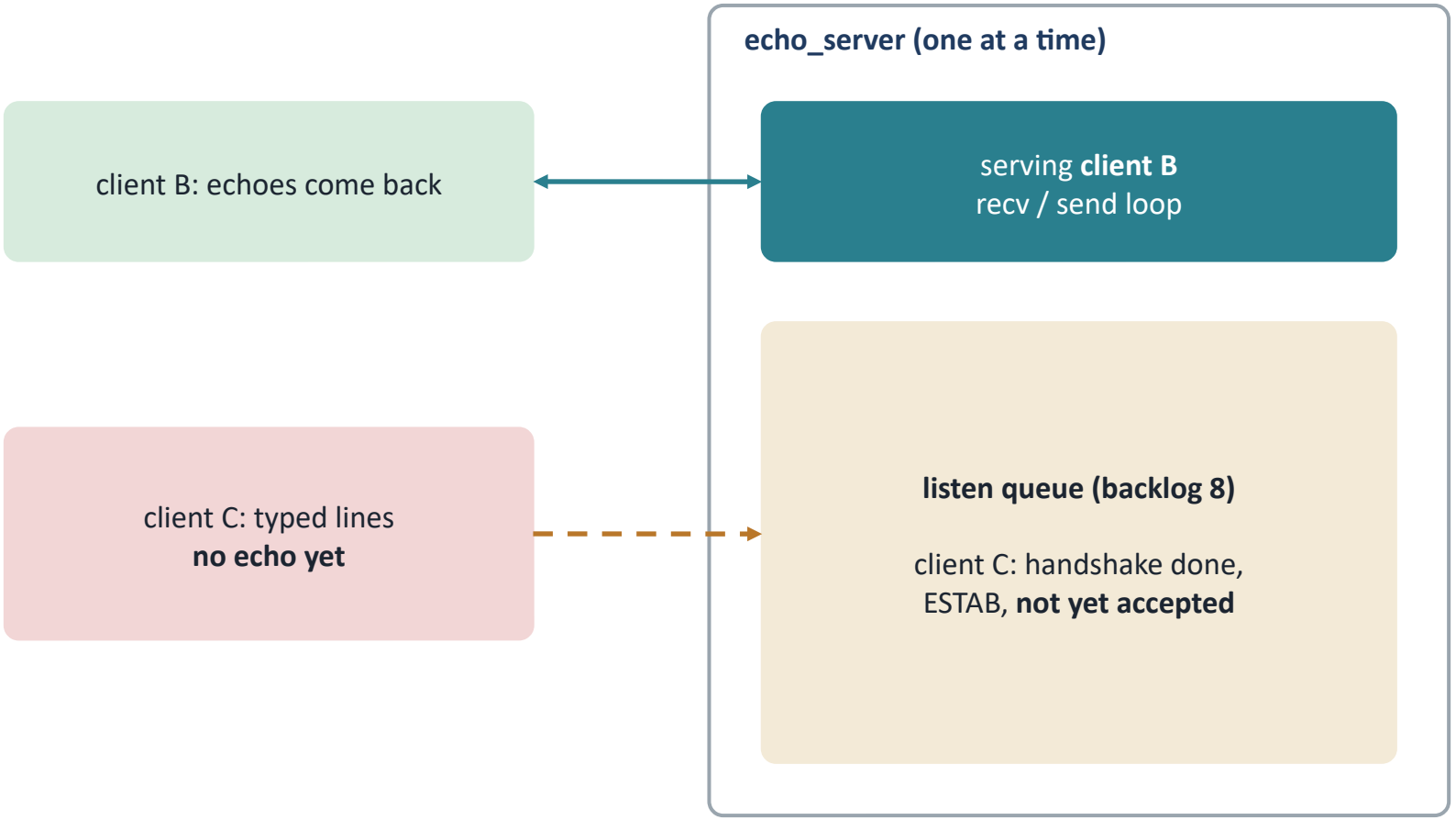
SO_REUSEADDR (socket option: reuse address)
lets the server restart at once

htons(port) (host to network short)
port in network byte order

1 Q5 · the server prints each client's address and **ephemeral** port, taken from accept().

Key idea socket → bind → listen → accept, then serve the client until recv returns 0, close, and accept again.

Task 3 · Why the second client waits



1. Q6 · ss -tn shows C as ESTAB: the kernel completed the handshake.
2. Laboratory 5 builds **concurrent** servers.

Key idea The kernel accepts connections into a queue; an iterative server takes them out one at a time.

ss (socket statistics)

Task 3 · Q7 · Forgetting htons

5000 = **0x1388**

stored as (little-endian)

88	13

read as network order

88	13

0x8813 = 34 835

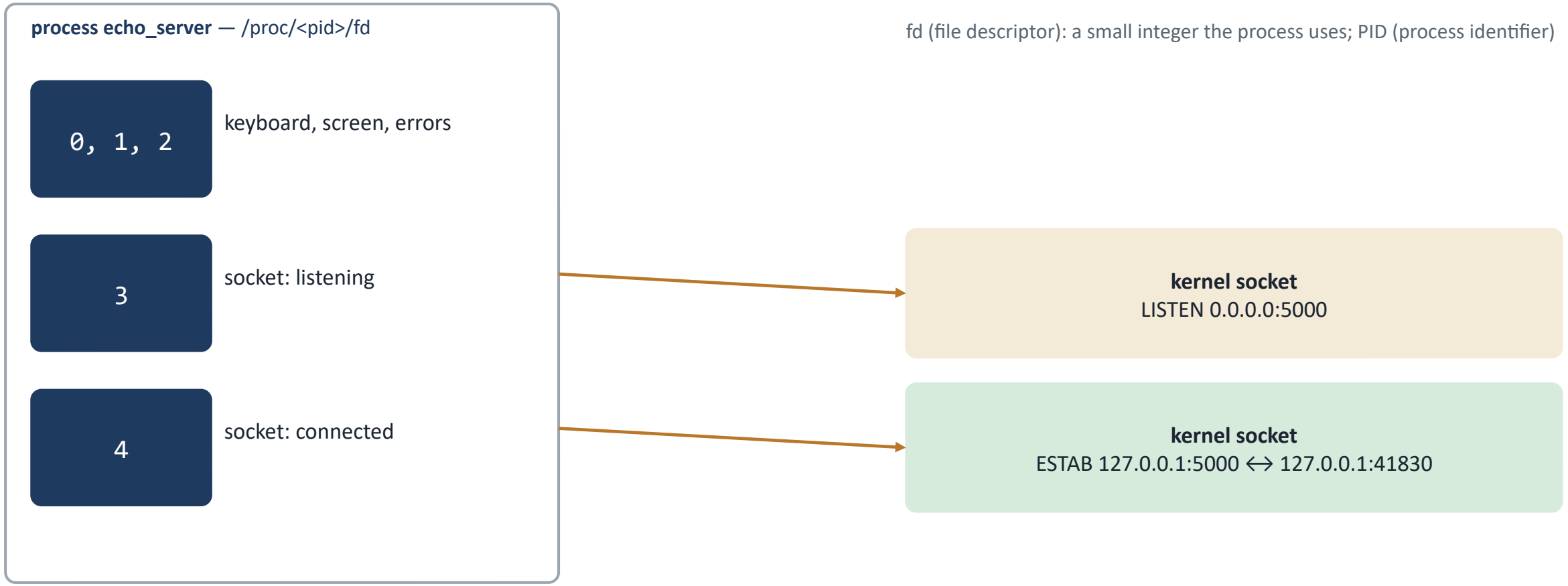
`ss -tln` shows the server listening on **34835**

a client connecting to 5000 gets “Connection refused”

Key idea htons() (host to network short) is not optional: without it the server listens on the wrong port.

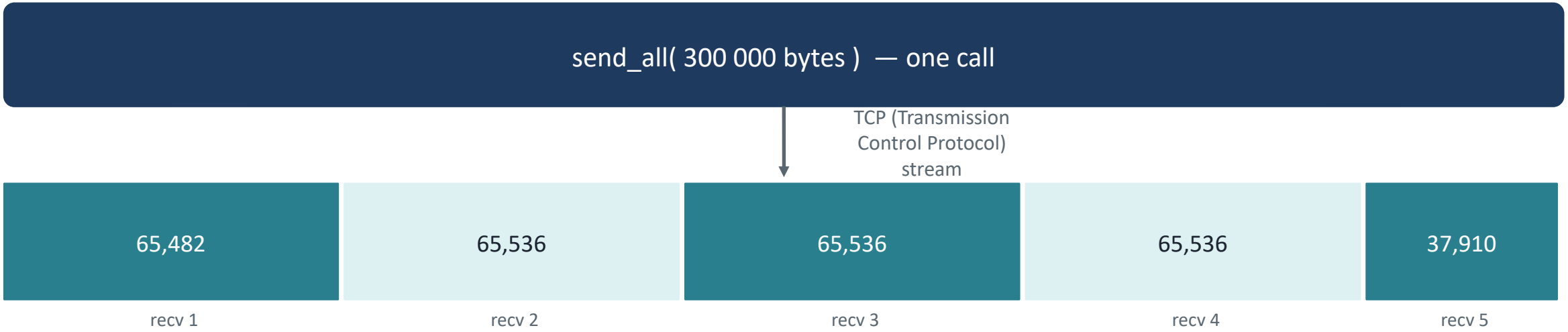
ss (socket statistics)

Task 4 · Sockets as the operating system sees them



Key idea Q8 · With one client: two sockets, fd 3 (LISTEN) and fd 4 (ESTAB).

Task 5 · One send of 300 000 bytes, five receives



1. Q9 · one send, **several** receives; the sizes change from run to run and are capped by the server's buffer (65 536 bytes).

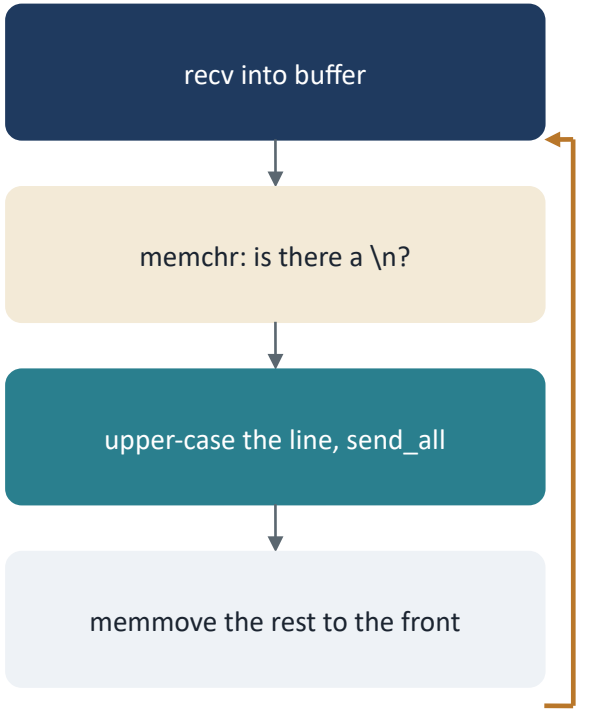
2. Q10 · with a 7-byte buffer, "hello world\n" arrives as 7 + 5 — and the client still gets it whole thanks to recv_all.

Key idea Never assume one recv = one send: loop until you have the bytes your protocol expects.

Task 6 · Framing with lines: the upper-case server

What one recv() may contain

part of a line	`ab`	→ wait for more
exactly one line	`abc\n`	→ answer ABC
several lines	`abc\n` `def\n` `gh`	→ answer two lines, keep “gh”

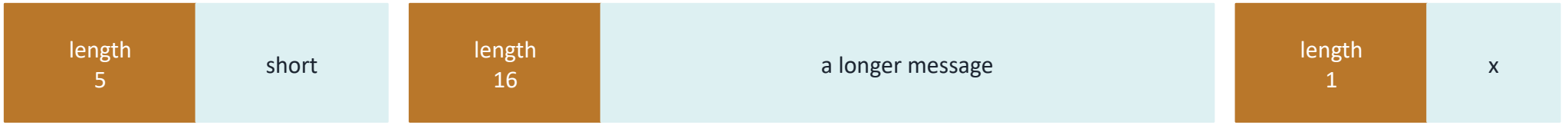


1 Q11 · “ghi” has no \n: it is never a complete line, so never answered.

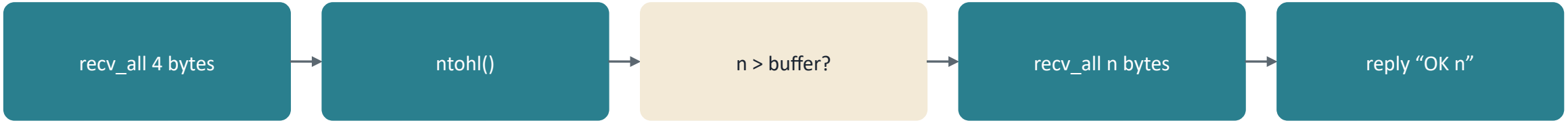
2 Q12 · a line > 4096 bytes fills the buffer: the server replies ERR and closes.

Key idea A line-based protocol must buffer: a line can be split across receives, or several lines can arrive together.

Task 7 · Framing with a length prefix



on the wire: 4 + 5 + 4 + 16 + 4 + 1 = **34 bytes**



1 Q13 · htonl() (host to network long) makes the length readable on any machine.

2 Q14 · a file over 1 MB exceeds the server’s buffer: recv_msg rejects it.

Key idea Length first, payload second: framing that works for any data, including binary.

Task 8 · Breaking things on purpose

1 Connection refused

client SYN (synchronize) → host answers RST (reset)

nothing listens on that port

2 Address already in use

bind() fails for about a minute

the old connection is in TIME_WAIT; SO_REUSEADDR (socket option: reuse address) avoids it

3 Peer closed

client FIN (finish) → recv() returns 0

the normal end of a stream

4 Broken pipe

send() to a closed connection

SIGPIPE (signal: broken pipe) kills the program; ignore it and send fails with EPIPE (error: broken pipe)

Key idea Q15–Q16 · Each error message names a TCP event: RST, TIME_WAIT, FIN or a write to a dead connection.

TCP (Transmission Control Protocol)

The same programs in Python

C	Python
`socket(AF_INET, SOCK_STREAM)`	`socket.socket(AF_INET, SOCK_STREAM)`
`bind() + listen(8)`	`ls.bind((…)); ls.listen(8)`
`accept()`	`conn, addr = ls.accept()`
`recv() in a loop`	`while data := conn.recv(65536)`
`send_all()`	`conn.sendall(data)`
`close()`	`with … : (closes automatically)`

Key idea Same calls, shorter syntax: the stream behaviour — and the need to loop — is identical.

SOCK_STREAM (byte-stream socket) · AF_INET (address family Internet)

Part D · Laboratory 3 on Windows: Winsock

A · Lecture 3
The application layer

→

B · Seminar 3
Problems with solutions

→

C · Laboratory 3 · Linux
Sockets in C

→

D · Laboratory 3 · Windows
Sockets with Winsock

In this part

Toolchain · building with ws2_32

Task 1 · A listening socket in PowerShell

Task 4 · Sockets as Windows sees them

Tasks 6–7 · Framing, ported

Winsock vs POSIX: the differences

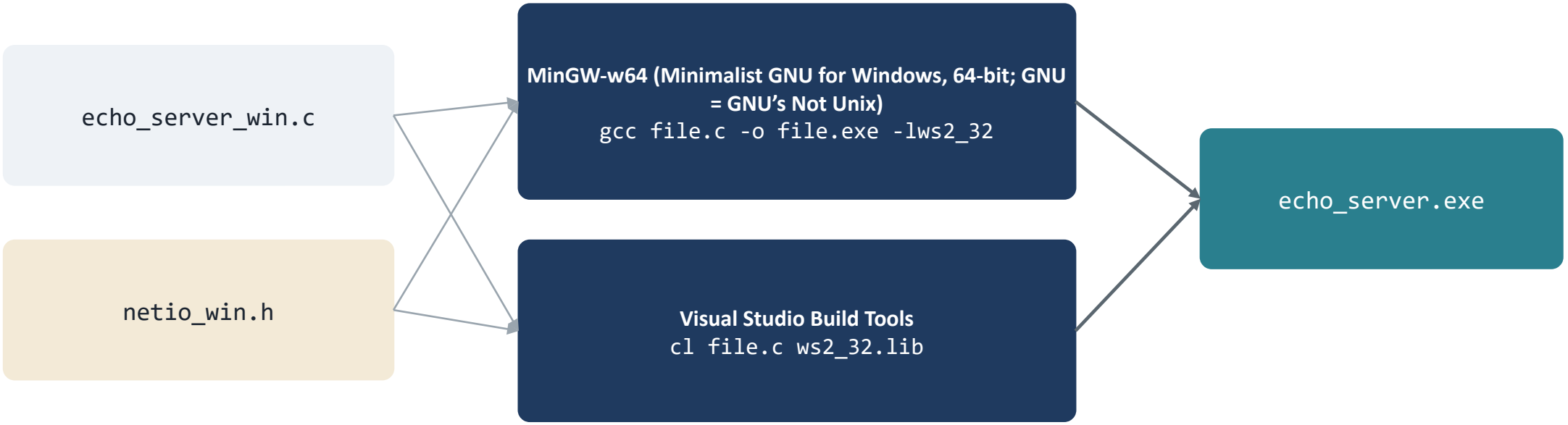
Tasks 2–3 · Client and server

Task 5 · A stream is not messages

Task 8 · Errors and SO_EXCLUSIVEADDRUSE

SO_EXCLUSIVEADDRUSE (socket option: exclusive address use) · POSIX (Portable Operating System Interface)

Toolchain · Building a Winsock program



bind to 127.0.0.1 (this machine only)
no Windows Firewall prompt — used in the lab

bind to all addresses (INADDR_ANY: any local address)
Windows asks for firewall permission the first time

Key idea Forgetting -lws2_32 gives "undefined reference" link errors; binding to 127.0.0.1 avoids the firewall prompt.

Winsock vs POSIX: what changes

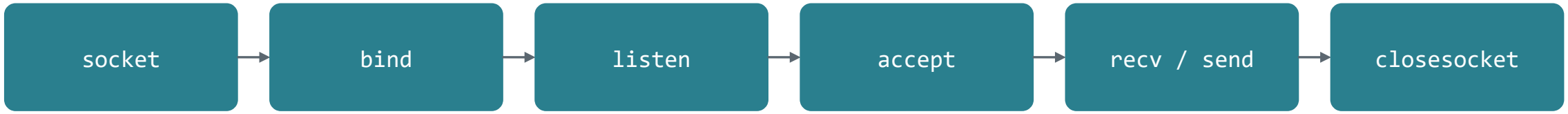
Linux	Windows
(nothing)	WSAStartup() … WSACleanup()
int fd	SOCKET s
-1 on failure	INVALID_SOCKET / SOCKET_ERROR
close(fd)	closesocket(s)
errno, perror()	WSAGetLastError()
read/write or recv/send	recv/send only, lengths are int
SIGPIPE (signal: broken pipe)	no signal; send fails

Key idea Every Winsock program starts with WSAStartup, closes sockets with closesocket, and reads errors with WSAGetLastError.

WSAGetLastError (last Windows Sockets error) · WSAStartup (starts Windows Sockets) · WSACleanup (stops Windows Sockets) · POSIX (Portable Operating System Interface) · fd (file descriptor)

The life of a Winsock program

WSAStartup(MAKEWORD(2, 2)) — load Winsock version 2.2



after each call: compare with **INVALID_SOCKET** or **SOCKET_ERROR**; on failure print **WSAGetLastError()**

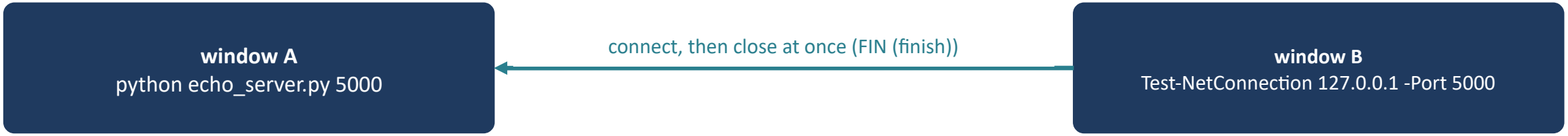
e.g. 10061 = connection refused · 10048 = address in use · 10054 = connection reset

WSACleanup() — release Winsock

Key idea Forget WSAStartup and every call fails with error 10093 (not initialised).

WSAGetLastError (last Windows Sockets error) · WSAStartup (starts Windows Sockets) · WSACleanup (stops Windows Sockets)

Task 1 · A listening socket seen from PowerShell



Get-NetTCPConnection before

```
Listen  127.0.0.1:5000
```

during the test

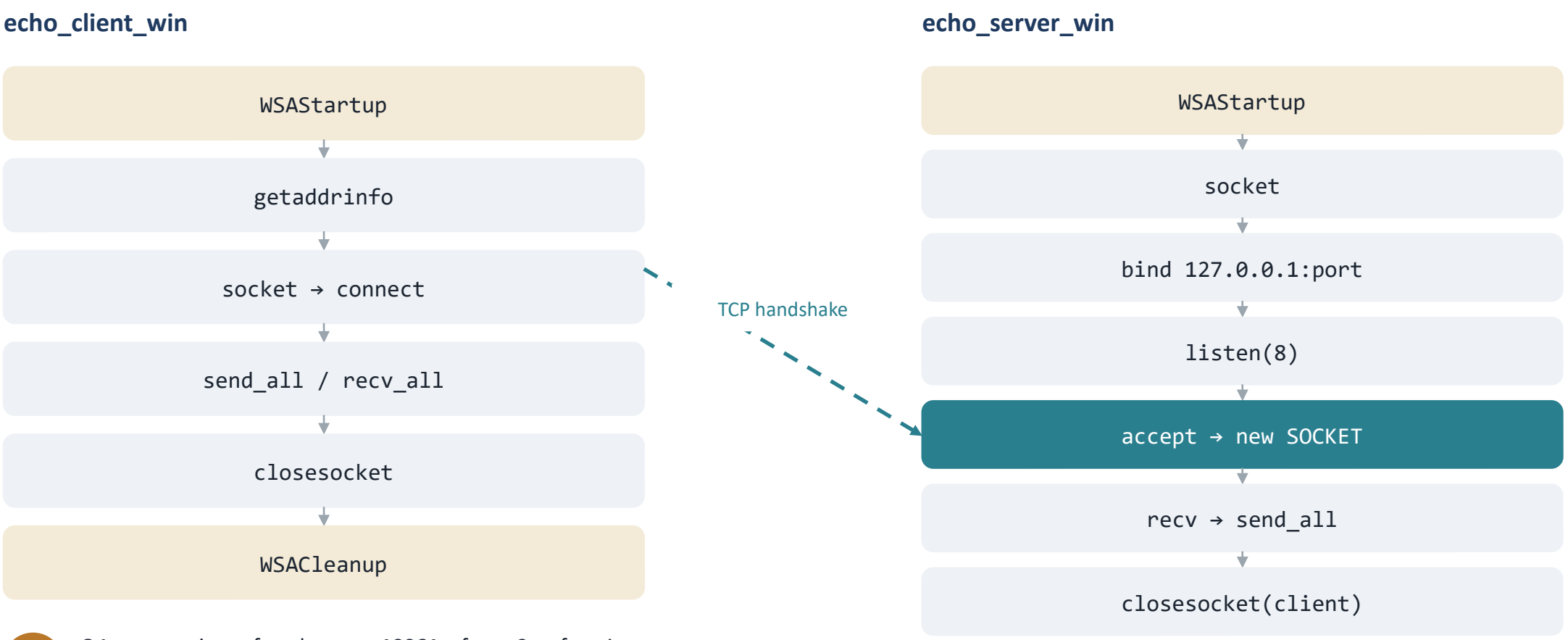
```
Established  127.0.0.1:5000 ⇔ 127.0.0.1:5xxxx
```

1 Q1–Q2 · the client port is chosen by Windows; the server prints “client closed after 0 bytes” when the FIN arrives.

Key idea Get-NetTCPConnection is the Windows view of ss: states, 4-tuples and the owning process.

ss (socket statistics)

Tasks 2–3 · Winsock client and server, side by side



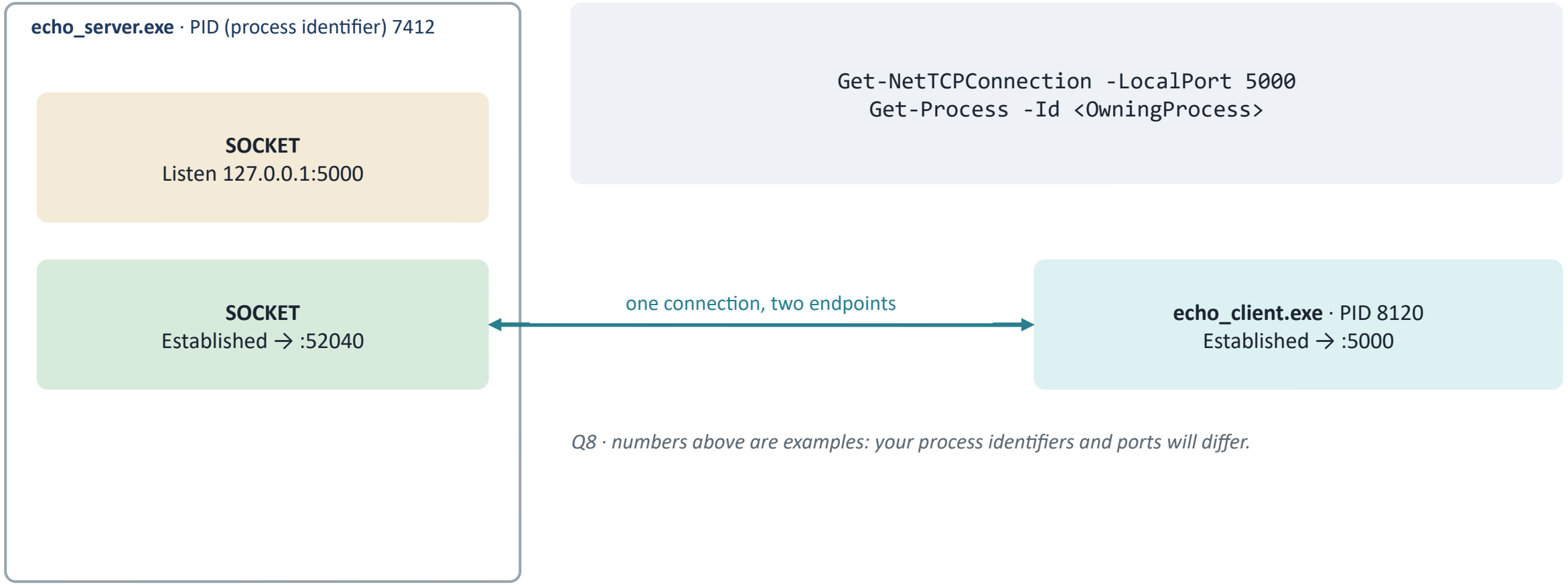
1

Q4 · connection refused = error **10061**, after ≈ 2 s of retries.

Key idea Q5–Q7 · Same steps as on Linux; the second client waits in the queue; without htons the port is wrong.

WSAStartup (starts Windows Sockets) · WSACleanup (stops Windows Sockets) · htons (host to network short) · TCP (Transmission Control Protocol)

Task 4 · Sockets as Windows sees them




```
Get-NetTCPConnection -LocalPort 5000
  Get-Process -Id <OwningProcess>
```

Q8 · numbers above are examples: your process identifiers and ports will differ.

Key idea Each end of a connection is a socket owned by a process; PowerShell maps them with OwningProcess.

Task 5 · One write from PowerShell, several receives

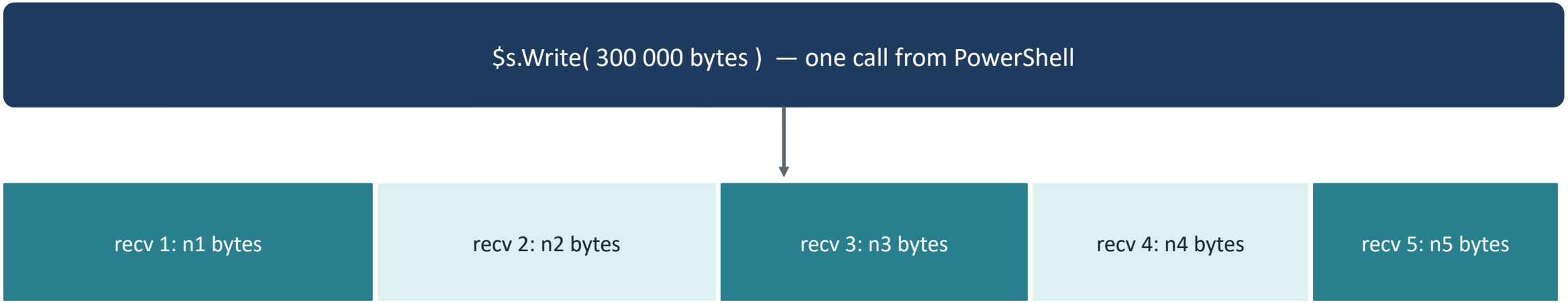


illustration: measure the real sizes yourself — they change between runs

1. Q9 · n1 + n2 + … = 300 000, but no single recv need equal the write.
2. Q10 · with a 7-byte buffer the server echoes in 7-byte pieces; the C client reassembles them with recv_all.

Key idea On Windows exactly as on Linux: TCP delivers a stream, not your writes.

TCP (Transmission Control Protocol)

Tasks 6–7 · Porting the framing servers to Winsock

unchanged

buffer + memchr for lines

send_msg / recv_msg for length prefixes

htonl / ntohl (host ↔ network long)

changes

WSAStartup / WSACleanup

int → SOCKET

close → closesocket

errno → WSAGetLastError()

link with ws2_32

1

Q11–Q14 · the same answers as on Linux: incomplete last line never answered; length prefix finds every boundary.

Key idea Framing is protocol logic, independent of the operating system; only the API spelling differs.

WSAGetLastError (last Windows Sockets error) · WSAStartup (starts Windows Sockets) · WSACleanup (stops Windows Sockets) · htonl (host to network long) · ntohl (network to host long) · API (application programming interface)

Task 8 · Errors on Windows, mapped to Linux

event	Windows	Linux
connection refused	10061 WSAECONNREFUSED, after ≈ 2 s	ECONNREFUSED (error: connection refused), at once
address in use	10048 WSAEADDRINUSE	EADDRINUSE (error: address in use)
peer killed	10054 WSAECONNRESET	ECONNRESET (error: connection reset)
write after close	10053 / 10054, no signal	SIGPIPE (signal: broken pipe) or EPIPE (error: broken pipe)

Key idea Q15 · Look up the number, then name the TCP event: RST, FIN, or a port already taken.

WSAECONNREFUSED (Windows Sockets error: connection refused) · WSAEADDRINUSE (Windows Sockets error: address in use) · WSAECONNRESET (Windows Sockets error: connection reset) · TCP (Transmission Control Protocol) · FIN (finish)

Task 8 · Q16 · SO_REUSEADDR means something else on Windows

with SO_REUSEADDR on Windows

server A
port 5000

program B
port 5000 too!

Both bind the same port.

Which one receives the next client is not defined — B may **steal** A's connections.

with SO_EXCLUSIVEADDRUSE

server A
port 5000

program B
bind → 10048

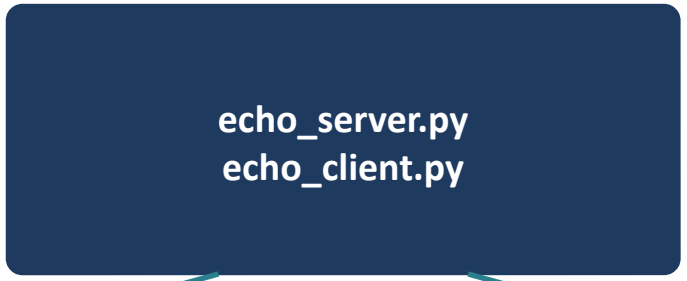
The port belongs to A alone.

Windows usually lets a **restarted** server bind again at once, so the Linux reason for SO_REUSEADDR does not apply.

Key idea Same option name, opposite risk: on Windows protect server ports with SO_EXCLUSIVEADDRUSE.

SO_EXCLUSIVEADDRUSE (socket option: exclusive address use) · SO_REUSEADDR (socket option: reuse address)

One Python file, two operating systems



Linux
socket(), close(), errno
python3 echo_server.py 5000

Windows
Winsock: WSAStartup, closesocket
python echo_server.py 5000

1 recv_all is still needed: a recv may return fewer bytes than were sent, on both systems.

Key idea The socket concepts are portable; only the C spelling of the API is not.

WSAStartup (starts Windows Sockets) · API (application programming interface)

Next week

Lecture 4

cookies and state · web caching and conditional GET · HTTP/2 and HTTP/3 · content distribution networks

Seminar 4

caching and timing problems

Laboratory 4

UDP (User Datagram Protocol) sockets · byte order in practice

Key idea This week's framing and byte-order skills are exactly what next week's UDP programs need.

HTTP (HyperText Transfer Protocol)